



郑州大学  
Zhengzhou University

# 第二单元 分组密码

郑州大学网络空间安全学院

王志华

王志华



## 第2单元 分组密码

提纲

CONTENTS

2.1 分组密码概述

2.2 分组密码分析

2.3 DES

2.4 AES

2.5 SM4

2.6 祖冲之密码

2.7 其他分组密码

2.8 分组密码工作模式

2.2.2.2

### 2.4.1 AES描述

#### 1、AES概述

- ❖ DES存在的问题：
  - DES存在安全问题，而三重DES算法运行速度比较慢
  - 其次三重DES的分组长度为64位，就效率和安全性而言，分组长度应该更长
  - 由于DES要花费比较大的软硬件代价，效率不尽如人意
  - DES密钥较短不能抵抗密钥穷举攻击，于是更好的标准AES走向了前台

王卫华



## AES候选算法:

- AES:一种基于Rijndael加密算法的默认分组密码
- Serpent:AES竞赛最后一轮评审中的候选算法, 在安全性和效率之间做出了与AES略有不同的权衡, 选择了较慢但被认为更安全的设计
- Twofish: AES竞赛最后一轮评审中的候选算法, 基于早期的分组密码Blowfish而设计
- MARS
- RC6

王华



## AES概述:

- AES采用SPN结构，也是一个迭代分组密码；
- AES分组长度：128bits (16bytes) /192/256bits，AES密钥长度：128/192/256bits；
- AES的轮数 $N_r$ 依赖于密钥长度；
- AES算法中有些是以字节为单位进行运算的；
- AES算法中也有的的是以4个字节（即一个字）为单位进行运算的；
- AES将一个字节看作是在有限域 $GF(2^8)$  上的一个元素；
- AES将一个字看成是系数取自 $GF(2^8)$ ，并且次数小于4的多项式。
- AES的应用产品：移动硬盘，USB无线网卡，加密锁，指纹加密U盘，指纹加密防护盾等。

2022/3/19



## 2、AES发展史

- 1997年1月2日，为了代替行将退役的DES，美国国家标准技术研究所NIST开始了遴选DES替代者的工作；
- 1997年4月15日，NIST发起了征集AES(Advanced Encryption Standard)算法的活动，并专门成立了一个AES工作组。目的是为了确定一个非保密的、全球免费使用的分组密码算法，用于保护下一世纪政府的敏感信息；
- 1997年9月12日，在联邦登记处（FR）公布了征集AES算法的通告，要求如下：
  - AES比3DES快，
  - 至少和3DES一样安全，
  - 特别提出高级加密标准的分组长度为128位的对称分组密码，密钥长度支持128位、192位、256位；
  - 可在全世界范围内免费得到。

王华



## AES的要求:

- (1) AES是公开的;
- (2) AES为单钥体制分组密码;
- (3) AES的密钥长度可变, 可按需要增大;
- (4) AES适于用软件和硬件实现;
- (5) AES可以自由地使用, 或按符合美国国家标准 (ANST) 策略的条件使用;

2022/3/19



AES的衡量条件:

满足以上要求的AES算法，需按下述条件判断优劣

- a. 安全性
- b. 计算效率
- c. 内存要求
- d. 使用简便性
- e. 灵活性。

2022/3/19





## AES的发展史:

- 1998年6月15日, 在提交的21个算法中, 有15个满足所有必备条件;
- 1998年8月20日, NIST召开了第一次AES候选会议, 公布了满足候选要求的15个AES候选算法 (共收到21个算法) ;

| 序号 | 算法名称     | 申请者                                                                 |
|----|----------|---------------------------------------------------------------------|
| 1  | CAST-256 | Entrust Technologies, Inc.                                          |
| 2  | CRYPTON  | Future System, Inc.                                                 |
| 3  | DEAL     | Richard Outerbridge, Lars Knudsen                                   |
| 4  | DFC      | Centre National pour la Recherche Scientifique                      |
| 5  | E2       | Nippon Telegraph and Telephone Corporation                          |
| 6  | FROG     | TecApro International S. A.                                         |
| 7  | HPC      | Rich Schroeppel                                                     |
| 8  | LOKI97   | Lawrie Brown, Josef Pieprzyk, Jennifer Seberry                      |
| 9  | MAGENTA  | Deutsche Telekom AG                                                 |
| 10 | MARS     | IBM                                                                 |
| 11 | RC6      | RSA Laboratories                                                    |
| 12 | RIJNDAEL | Joan Daemen, Vincent Rijmen                                         |
| 13 | SAFER+   | Cylink Corporation                                                  |
| 14 | SERPENT  | Ross Anderson, Eli Biham, Lars Knudsen                              |
| 15 | TWOFISH  | B. Schneier, J. Kelsey, D. Whiting, D. Wagner, C. Hall, N. Ferguson |



## AES算法的评估准则:

- **安全性**: 算法的“安全性”无疑是最重要的, 如果一个算法被发现不安全就不会再被考虑。主要要求算法抗密码分析的强度, 稳定的数学基础, 算法输出的随机性, 与其他候选算法比较的相对安全性。
- **代价**: 各种实现的计算效率(速度和存储需求), 包括软件实现、硬件实现和智能卡实现。。
- **算法和实现特性**: 灵活性、硬件和软件适应性、算法的简单性等。算法的灵活性包括:
  - ① 处理的密钥和分组长度必须要超出最小的支持范围;
  - ② 在许多不同类型的环境中能够安全和有效地实现;
  - ③ 可以作为序列密码、杂凑算法实现, 并且可提供附加的密码服务。算法必须能够用软件和硬件两种方法实现, 并且有利于有效的固件实现。



## AES的发展史:

- 1999年3月22日, NIST举行了第二次候选会议, 公布了15个算法的讨论结果, 并从中选出了5个候选算法: MARS, RC6, Rijndael, SERPENT, Twofish;
- 2000年4月25日, NIST举行了第三次AES候选会议, 对第二轮的5个算法进行了讨论;
- 2000年10月2日, NIST公布了最终评选结果, 获胜者是 **Rijndael算法**;
  - Rijndael是比利时的Joan Daemen和Vincent Rijmen 提交的一个候选算法。
- 2001年2月28日, NIST宣布了关于AES的联邦信息处理标准的草案可供公众评判;
- 2001年11月26日, AES被采纳为标准;
- 2001年12月4日, 在美国联邦记录中作为FIPS 197公布。



~ ~ ~



## AES的趣事:

- AES 的遴选过程以其**公开性**和**国际性**闻名。
- 三次候选算法大会和官方请求公众评审为候选算法意见的反馈、公众讨论与分析提供了足够的机会，而且这一过程为置身其中的每一个人所称道。
- 15 个 AES 候选算法的作者代表着不同国家：澳大利亚、比利时、加拿大、哥斯达黎加、法国、德国、以色列、日本、韩国、挪威、英国及美国，这正表明了AES 的国际性。
- “第二次 AES 候选大会”在美国之外的意大利罗马举行。
- 最后，五个入围最终决赛的算法都被认为是安全的。  
Rijndael 之所以最后当选是由于它集安全性、性能、效率、可实现性及灵活性于一体，被认为优于其他四个决赛者。

王卫华





### 3、Ruijndael的数学基础

#### ■ 模运算法则：群、交换群、环、有限环

1. 对加法运算封闭:对任意的  $a, b \in \mathbb{Z}_m$ , 有  $a + b \in \mathbb{Z}_m$ 。
2. 加法运算满足交换律:对任意的  $a, b \in \mathbb{Z}_m$ , 有  $a + b = b + a$ 。
3. 加法运算满足结合律:对任意的  $a, b, c \in \mathbb{Z}_m$ , 有  $(a + b) + c = a + (b + c)$ 。
4. 0 是加法单位元:对任意的  $a \in \mathbb{Z}_m$ , 有  $a + 0 = 0 + a = a$ 。
5. 任意  $a \in \mathbb{Z}_m$  的加法逆元为  $m - a$ , 因为  $a + (m - a) = (m - a) + a = 0$ 。
6. 对乘法运算封闭:对任意的  $a, b \in \mathbb{Z}_m$ , 有  $ab \in \mathbb{Z}_m$ 。
7. 乘法运算满足交换律:对任意的  $a, b \in \mathbb{Z}_m$ , 有  $ab = ba$ 。
8. 乘法运算满足结合律:对任意的  $a, b, c \in \mathbb{Z}_m$ , 有  $(ab)c = a(bc)$ 。
9. 1 是乘法单位元:对任意的  $a \in \mathbb{Z}_m$ , 有  $a \times 1 = 1 \times a = a$ 。
10. 乘法和加法之间存在分配律:对任意的  $a, b, c \in \mathbb{Z}_m$ , 有  $(a + b)c = (ac) + (bc)$ ,  $a(b + c) = (ab) + (ac)$ 。

性质 1、3 ~ 5, 说明  $\mathbb{Z}_m$  关于其上定义加法运算构成一个群; 若再加上性质 2, 则构成一个交换群(阿贝尔群)。

性质 1 ~ 10, 说明  $\mathbb{Z}_m$  是一个环。

普通加法和乘法的全体整数  $\mathbb{Z}$ 、全体实数  $\mathbb{R}$ 、全体复数  $\mathbb{C}$  等, 这些环均是无限环。但本书中我们关心的环都是有限环。



## ■ 模运算法则：域

域明确的满足如下性质：

### 在加法和乘法上封闭

对所有属于 $F$ 的 $a, b$ ,  $a + b$ 和 $a * b$ 属于 $F$ （另一种说法：加法和乘法是 $F$ 上的二元运算）。

### 加法和乘法符合结合律

对所有属于 $F$ 的 $a, b, c$ ,  $(a + b) + c = a + (b + c)$ ,  $(a * b) * c = a * (b * c)$

### 加法和乘法符合交换律

对所有属于 $F$ 的 $a, b$ ,  $a + b = b + a$ ,  $a * b = b * a$

### 符合乘法对加法的分配律

对所有属于 $F$ 的 $a, b, c$ ,  $a * (b + c) = (a * b) + (a * c)$

### 存在加法单位

在 $F$ 中有元素 $0$ ，使得所有属于 $F$ 的 $a$ ,  $a + 0 = a$

### 存在乘法单位

在 $F$ 中有不同于 $0$ 的元素 $1$ ，使得所有属于 $F$ 的 $a$ ,  $a * 1 = a$

### 存在加法逆元

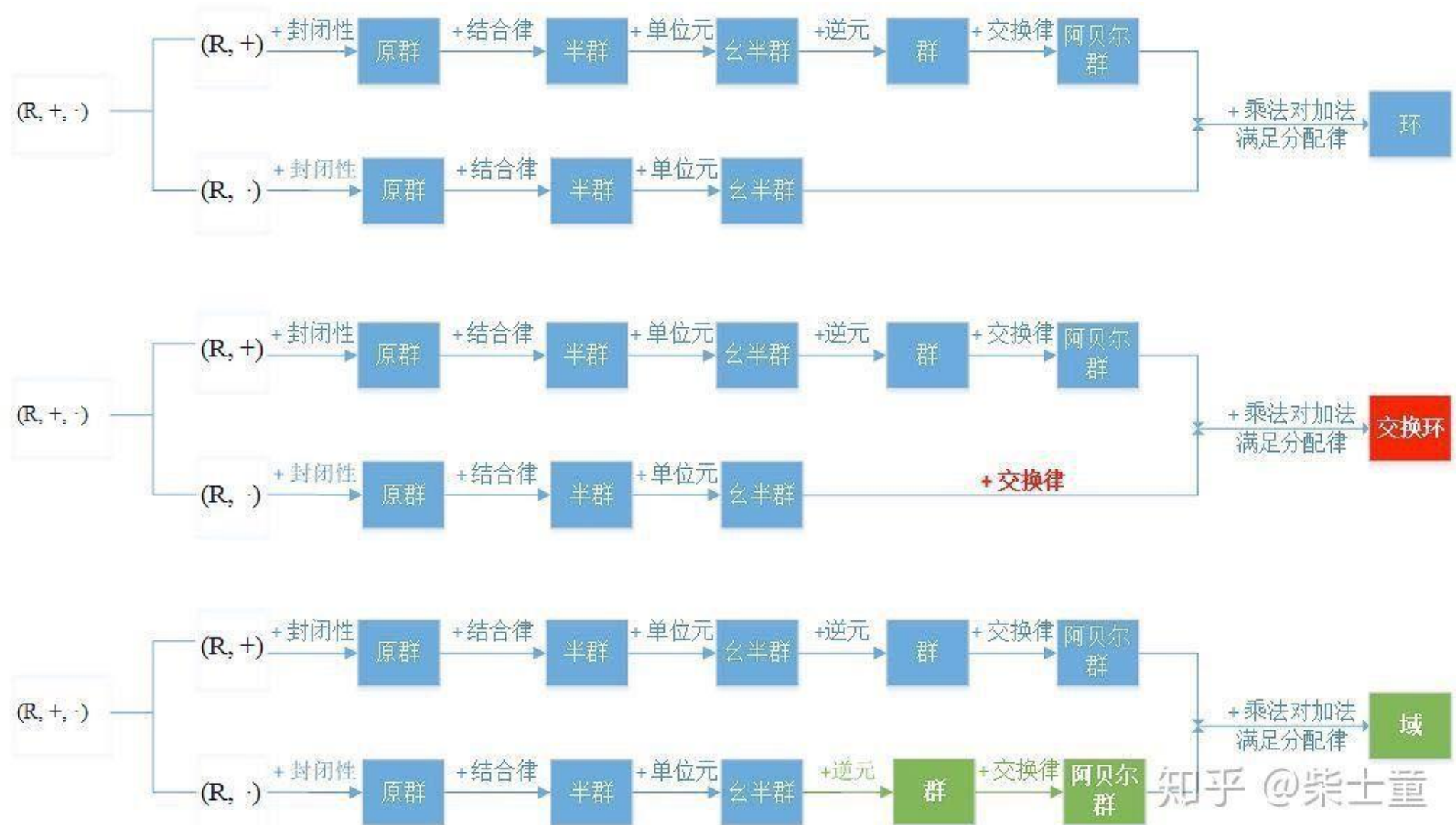
对所有属于 $F$ 的 $a$ ，存在 $-a$ 使得 $a + (-a) = 0$

### 存在乘法逆元

对所有 $a \neq 0$ ，存在元素 $a^{-1}$ 使得 $a * a^{-1} = 1$



# 模运算法则：群、交换群、环、有限环、域



2022/3/19





有限域:

## 有限域 $GF(2^8)$ 的多项式表示

- 有限域中的元素可用多种不同的方式表示。
- 对于任意素数（如2）的方幂（如 $2^n$ ），都有惟一的一个有限域（域中元素个数为 $2^n$ ），因此 $GF(2^8)$ 的所有表示是同构的，但不同的表示方法会影响到 $GF(2^8)$ 上运算的复杂度，本算法采用传统的**多项式表示法**(系数在 $GF(2)$ 域上的多项式)：

将 $b_7b_6b_5b_4b_3b_2b_1b_0$  构成的**字节**  $b$  看成系数在  $\{0,1\}$  中的多项式。

例如：十六进制数‘57’对应的二进制为01010111，看成一个字节，对应的多项式为：

$$x^6 + x^4 + x^2 + x + 1$$

王华



有限域:

● 有限域 $GF(2^8)$ 的多项式（字节）表示

➤ 下述表达式彼此等价:

$$(x^6 + x^4 + x^2 + x + 1) + (x^7 + x + 1) = x^7 + x^6 + x^4 + x^2$$

----- (多项式记法)

$$\{01010111\} \oplus \{10000011\} = \{11010100\}$$

----- (二进制记法)

$$\{57\} \oplus \{83\} = \{d4\}$$

----- (十六进制记法)

王卫华



系数在 $GF(2^8)$ 上的多项式:

● 系数在 $GF(2^8)$ 上的多项式的表示: 字运算

- 系数在有限域 $GF(2^8)$ 上的多项式**系统定义**为系数取自 $GF(2^8)$ 元素的多项式。
- **4个字节(1个字)构成的向量**可以表示为系数**在 $GF(2^8)$ 上的次数小于4**的多项式。

[https://www.thea.cn/xyk\\_zl\\_713803-1.htm](https://www.thea.cn/xyk_zl_713803-1.htm)

王卫华



有限域:

## 有限域 $GF(2^8)$ 上的加法：字节运算

- AES的理论基础定义在 $GF(2^8)$ 上，其基本的运算有三种，分别为加法，乘法和x乘。
- 在多项式表示中， $GF(2^8)$ 上两个元素的和仍然是一个次数不超过7的多项式，其系数等于两个元素对应系数的模2加（比特异或）。

例如：'57'+ '83'='D4'，用多项式表示为

$$(x^6+x^4+x^2+x+1)+(x^7+x+1)=x^7+x^6+x^4+x^2$$

用二进制表示为

$$01010111+10000011=11010100$$

由于每个元素的加法逆元等于自己，所以减法和加法相同。

王卫华



有限域:

## 有限域 $GF(2^8)$ 上的乘法：字节运算

- 要计算 $GF(2^8)$ 上的乘法，必须先确定一个 $GF(2)$ 上的8次不可约多项式； $GF(2^8)$ 上两个元素的乘积就是这两个多项式的模乘（以这个8次不可约多项式为模）。在Rijndael密码中，这个8次不可约多项式确定为：

$$m(x) = x^8 + x^4 + x^3 + x + 1$$

它的十六进制表示为‘11B’。

例如：‘57’ · ‘83’ = ‘C1’ 可表示为以下的多项式乘法

$$(x^6 + x^4 + x^2 + x + 1)(x^7 + x + 1) = x^7 + x^6 + 1 \pmod{m(x)}$$

王华



有限域:

● 有限域 $GF(2^8)$ 上的乘法: 字节运算



$$0x57 \cdot 0x83 = 0xc1$$

$$(x^6 + x^4 + x^2 + x + 1)(x^7 + x + 1)$$

$$= x^{13} + x^{11} + x^9 + x^8 + x^7 + x^7 + x^5 + x^3 \\ + x^2 + x + x^6 + x^4 + x^2 + x + 1$$

$$= x^{13} + x^{11} + x^9 + x^8 + x^6 + x^5 + x^4 + x^3 + 1$$

$$x^{13} + x^{11} + x^9 + x^8 + x^6 + x^5 + x^4 + x^3 + 1$$

$$\text{mod } x^8 + x^4 + x^3 + x + 1 = x^7 + x^6 + 1$$

2022





有限域:

## 有限域 $GF(2^8)$ 上的乘法：字节运算

- 以上定义的乘法满足**交换律**，且有**单位元‘01’**。另外，对任何次数小于8的多项式 $b(x)$ ，可用**扩展的欧几里德算法**得：

$$b(x)a(x)+m(x)c(x)=1$$

即： $a(x)b(x)=1 \bmod m(x)$ ，因此 $a(x)$ 是 $b(x)$ 的**乘法逆元**。再者，乘法还满足**分配律**：

$$a(x) \odot (b(x) + c(x)) = a(x) \odot b(x) + a(x) \odot c(x)$$

所以，256个**字节表示的值**构成的集合，在以上定义的和乘法运算下，有有限域 $GF(2^8)$ 的结构。

**注：**非0元的乘法满足Abel群，加法满足Abel群， $+$ ， $\times$ 满足分配率。

王卫华



有限域:

## 有限域 $GF(2^8)$ 上的x乘法：字节运算

➤  $GF(2^8)$ 上还定义了一个运算，称之为**x乘法**，其定义为：

$$x \cdot b(x) = b_7 x^8 + b_6 x^7 + b_5 x^6 + b_4 x^5 + b_3 x^4 + b_2 x^3 + b_1 x^2 + b_0 x \pmod{m(x)}$$

如果  $b_7 = 0$ ，求模结果不变；否则如果  $b_7 = 1$ ，则为乘积结果减去  $m(x)$ ，即**求乘积结果与  $m(x)$  的异或**。

➤ 由此得出**x（十六进制‘02’）乘  $b(x)$** 可以先对  $b(x)$  在字节内左移一位（最后一位补0），若  $b_7 = 1$  则再与‘1B’（其二进制为**00011011**）做逐比特异或来实现，该运算记  $b = xtime(a)$ 。

➤ 在专用芯片中， $xtime$  只需4个异或。 $x$  的幂乘运算可以重复应用  $xtime$  来实现。而任意常数乘法可以通过对中间结果相加实现。

王华



有限域:

## 有限域 $GF(2^8)$ 上的x乘法: 字节运算

例如: '57' · '13' 可如下实现:

$$'57' \cdot '02' = \text{xtime}(57) = 'AE' ;$$

$$'57' \cdot '04' = \text{xtime}(AE) = '47' ;$$

$$'57' \cdot '08' = \text{xtime}(47) = '8E' ;$$

$$'57' \cdot '10' = \text{xtime}(8E) = '07' ;$$

$$\begin{aligned} '57' \cdot '13' &= '57' \cdot ('01' \oplus '02' \oplus '10' ) \\ &= '57' \oplus 'AE' \oplus '07' = 'FE' . \end{aligned}$$

王华



系数在 $GF(2^8)$ 上的多项式:

## ● 系数在 $GF(2^8)$ 上的多项式的表示和运算：4字节运算

- 4个字节构成的向量可以表示为系数在 $GF(2^8)$ 上的次数小于4的多项式。
- 多项式的加法就是对应系数相加；换句话说，多项式的加法就是4字节向量的逐比特异或。
- 多项式的乘法运算必须要取模  $M(x)=x^4+1$ ，这样使得次数小于4的多项式的乘积仍然是一个次数小于4的多项式。
- 将多项式的模乘运算记为  $\otimes$ 。

设  $a(x) = a_3x^3 + a_2x^2 + a_1x + a_0$   $b(x) = b_3x^3 + b_2x^2 + b_1x + b_0$  则  
 $a(x) \otimes b(x) = c_3x^3 + c_2x^2 + c_1x + c_0$ 。由于  $x^j \bmod (x^4+1) = x^{j \bmod 4}$ ,

所以，有以下:

王华



系数在 $GF(2^8)$ 上的多项式:

● 系数在 $GF(2^8)$ 上的多项式的运算

$$c_0 = a_0b_0 \oplus a_3b_1 \oplus a_2b_2 \oplus a_1b_3;$$

$$c_1 = a_1b_0 \oplus a_0b_1 \oplus a_3b_2 \oplus a_2b_3;$$

$$c_2 = a_2b_0 \oplus a_1b_1 \oplus a_0b_2 \oplus a_3b_3;$$

$$c_3 = a_3b_0 \oplus a_2b_1 \oplus a_1b_2 \oplus a_0b_3。$$

可将上述计算表示为

$$\begin{pmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \end{pmatrix} = \begin{pmatrix} a_0 & a_3 & a_2 & a_1 \\ a_1 & a_0 & a_3 & a_2 \\ a_2 & a_1 & a_0 & a_3 \\ a_3 & a_2 & a_1 & a_0 \end{pmatrix} \begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{pmatrix}$$

2022



系数在 $GF(2^8)$ 上的多项式:

## ● 系数在 $GF(2^8)$ 上的多项式的运算

- 注意到  $M(x)$  不是  $GF(2^8)$  上的不可约多项式 (甚至也不是  $GF(2)$  上的不可约多项式), 因此非0多项式的这种乘法 **不是群运算**。
- 不过在Rijndael密码中, 对多项式  $b(x)$ , 这种乘法运算只限于乘一个固定的 **有逆元的多项式**  $a(x) = a_3x^3 + a_2x^2 + a_1x + a_0$ 。我们有如下的定理。

**定理3-1** 系数在 $GF(2^8)$ 上的多项式  $a_3x^3 + a_2x^2 + a_1x + a_0$  是模 $x^4+1$ 可逆的, 当且仅当**矩阵**

$$\begin{pmatrix} a_0 & a_3 & a_2 & a_1 \\ a_1 & a_0 & a_3 & a_2 \\ a_2 & a_1 & a_0 & a_3 \\ a_3 & a_2 & a_1 & a_0 \end{pmatrix}$$

**在 $GF(2^8)$ 上可逆。**

2022/3/19



系数在 $GF(2^8)$ 上的多项式:

## ● 系数在 $GF(2^8)$ 上的多项式的运算

证明  $a_3x^3+a_2x^2+a_1x+a_0$ 是模 $x^4+1$ 可逆的, 当且仅当存在多项式 $h_3x^3+h_2x^2+h_1x+h_0$ 使得

$$(a_3x^3+a_2x^2+a_1x+a_0)(h_3x^3+h_2x^2+h_1x+h_0)=1(\text{mod } x^4+1)$$

因此有

$$(a_3x^3+a_2x^2+a_1x+a_0)(h_2x^3+h_1x^2+h_0x+h_3)=x(\text{mod } x^4+1)$$

$$(a_3x^3+a_2x^2+a_1x+a_0)(h_1x^3+h_0x^2+h_3x+h_2)=x^2(\text{mod } x^4+1)$$

$$(a_3x^3+a_2x^2+a_1x+a_0)(h_0x^3+h_3x^2+h_2x+h_1)=x^3(\text{mod } x^4+1);$$

将以上关系写成矩阵形式即得

$$\begin{pmatrix} a_0 & a_3 & a_2 & a_1 \\ a_1 & a_0 & a_3 & a_2 \\ a_2 & a_1 & a_0 & a_3 \\ a_3 & a_2 & a_1 & a_0 \end{pmatrix} \begin{pmatrix} h_0 & h_3 & h_2 & h_1 \\ h_1 & h_0 & h_3 & h_2 \\ h_2 & h_1 & h_0 & h_3 \\ h_3 & h_2 & h_1 & h_0 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

2022/3/19





系数在 $GF(2^8)$ 上的多项式:

● 系数在 $GF(2^8)$ 上的多项式上的x模乘运算

$c(x) = x \otimes b(x)$  定义为 $x$ 与 $b(x)$ 的模 $x^4+1$ 乘法, 即

$$c(x) = x \otimes b(x) = b_2x^3 + b_1x^2 + b_0x + b_3$$

其矩阵表示中, 除 $a_1 = '01'$  外, 其它所有 $a_i = '00'$ , 即

$$\begin{pmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \end{pmatrix} = \begin{pmatrix} 00 & 00 & 00 & 01 \\ 01 & 00 & 00 & 00 \\ 00 & 01 & 00 & 00 \\ 00 & 00 & 01 & 00 \end{pmatrix} \begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{pmatrix}$$

因此,  $x$  (或 $x$ 的幂) 模乘多项式相当于对字节构成的向量进行字节循环移位。

王华

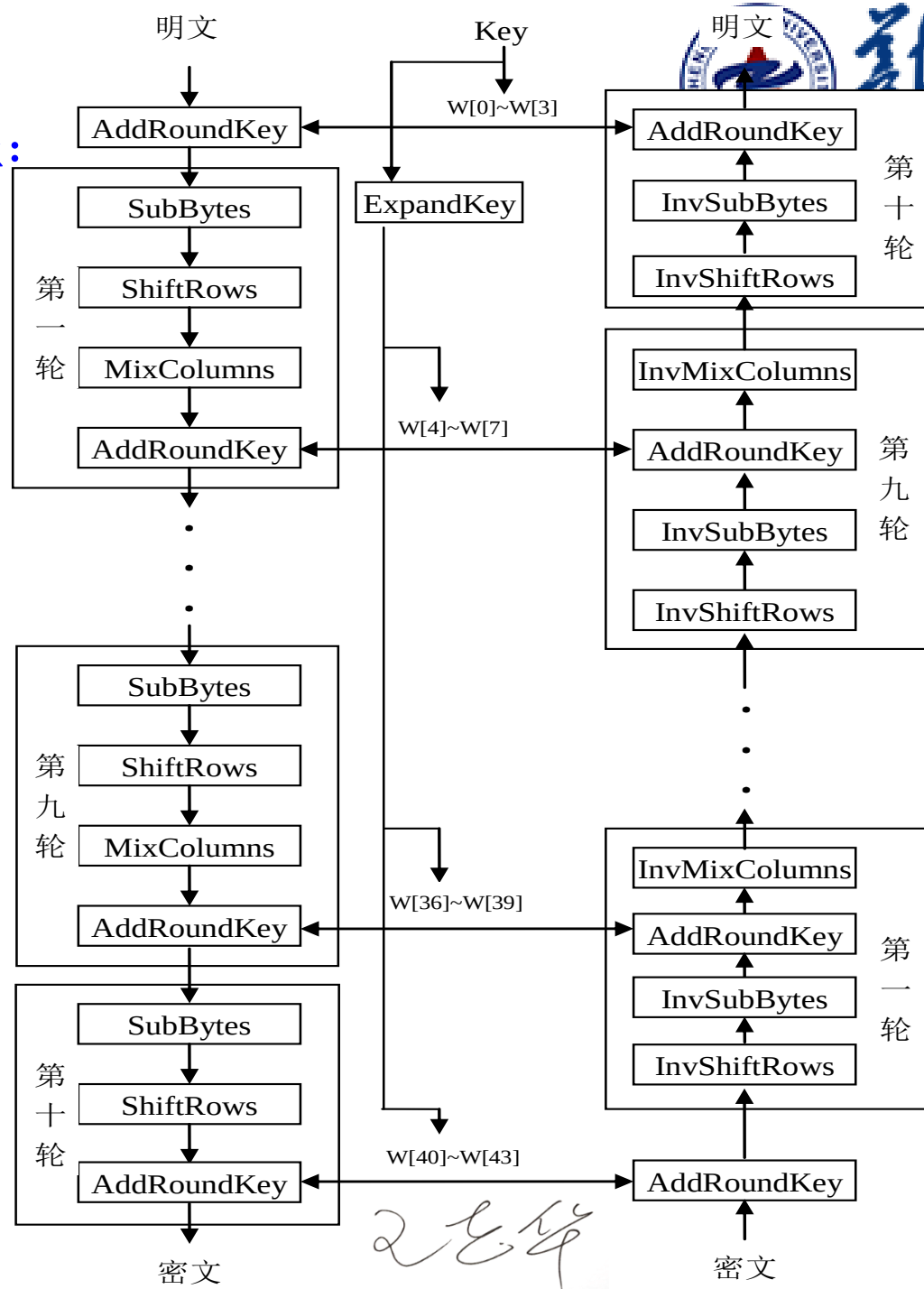


## 4、Rijndael的设计思路及特点

- Rijndael密码的设计力求满足以下3条标准：
  - 抵抗所有已知的攻击。
  - 在多个平台上速度快，编码紧凑。
  - 设计简单。

2022/3/19

# AES算法过程:



2022/3/19



## AES算法特点:

1. AES的加解密算法、密钥生成算法都是以State为输入。

2. 加密时，首先进行“轮密钥加”算法，然后重复9轮基本密码模块的迭代算法，每一轮有字节代换，行移位、列混淆和轮密钥加四个算法：最后进行第10轮运算，它与前面9轮比一样的地方，就是少了列混淆算法。

3. 解密时，是执行加密的逆过程，但要注意在解密时，字节代换、行移位和列混淆三种运算是执行逆过程，算法设计完全不一样，只有轮密钥加算法和加密一样。

王华



## AES算法过程:

📖 密钥加变换：即将密钥与输入状态进行字节异或，表示为  $\text{AddRoundKey}(\text{State}, w)$ ，其中  $\text{State}$  是加密过程中的一个状态， $w$  是层密钥。

📖 行变换：将  $\text{State}$  作为矩阵，进行行变换，表示为  $\text{ShiftRow}(\text{State})$ 。

📖 列变换：将  $\text{State}$  作为矩阵，进行列变换，表示为  $\text{MixColumn}(\text{State})$ 。

📖 字节变换：对  $\text{State}$  的每个字节（8位）作一个非线性变换，表示为  $\text{SubByte}(\text{State})$ 。

王华



## Rijndael的轮函数设计思想:

- Rijndael的**轮函数**是由三个不同的**可逆均匀变换**组成的，称它们为**三个“层”**。
  - 所谓“**均匀变换**”是指状态的每个比特都是用类似的方法进行处理的；
  - **不同层的特定选择**大部分是建立在“**宽轨迹策略**”的应用基础上的；
  - “**宽轨迹策略**”就是提供抗线性密码分析和差分密码分析能力的一种设计。

王卫华



## Rijndael的轮函数设计思想：宽轨迹策略

- 为实现**宽轨迹策略**，轮函数三个层中的每一层都有它自己的功能。
  - **线性混合层**：确保多轮之上的高度扩散；
  - **非线性层**：将具有最优的“最坏情况非线性特性”的S盒并行使用；
  - **密钥加层**：单轮子密钥简单地异或到中间状态上，实现一次性掩盖。

王华



## Rijndael的设计思想

- 在第一轮之前，用了一个**初始密钥加层**，其目的是在不知道密钥的情况下，对最后一个密钥加层以后的任一层（或者是当进行已知明文攻击时，对第一个密钥加层以前的任一层）可简单地剥去，因此初始密钥加层对密码的安全性无任何意义。
  - 许多密码的设计中都在轮变换之前和之后用了密钥加层，如**IDEA**、SAFER和Blowfish。
- 为了使加密算法和解密算法在结构上更加**接近**，最后一轮的**线性混合层**与前面各轮的线性混合层不同，这类似于DES的最后一轮不做左右交换一样。
  - 可以证明这种设计不以任何方式提高或降低该密码的安全性。

王卫华



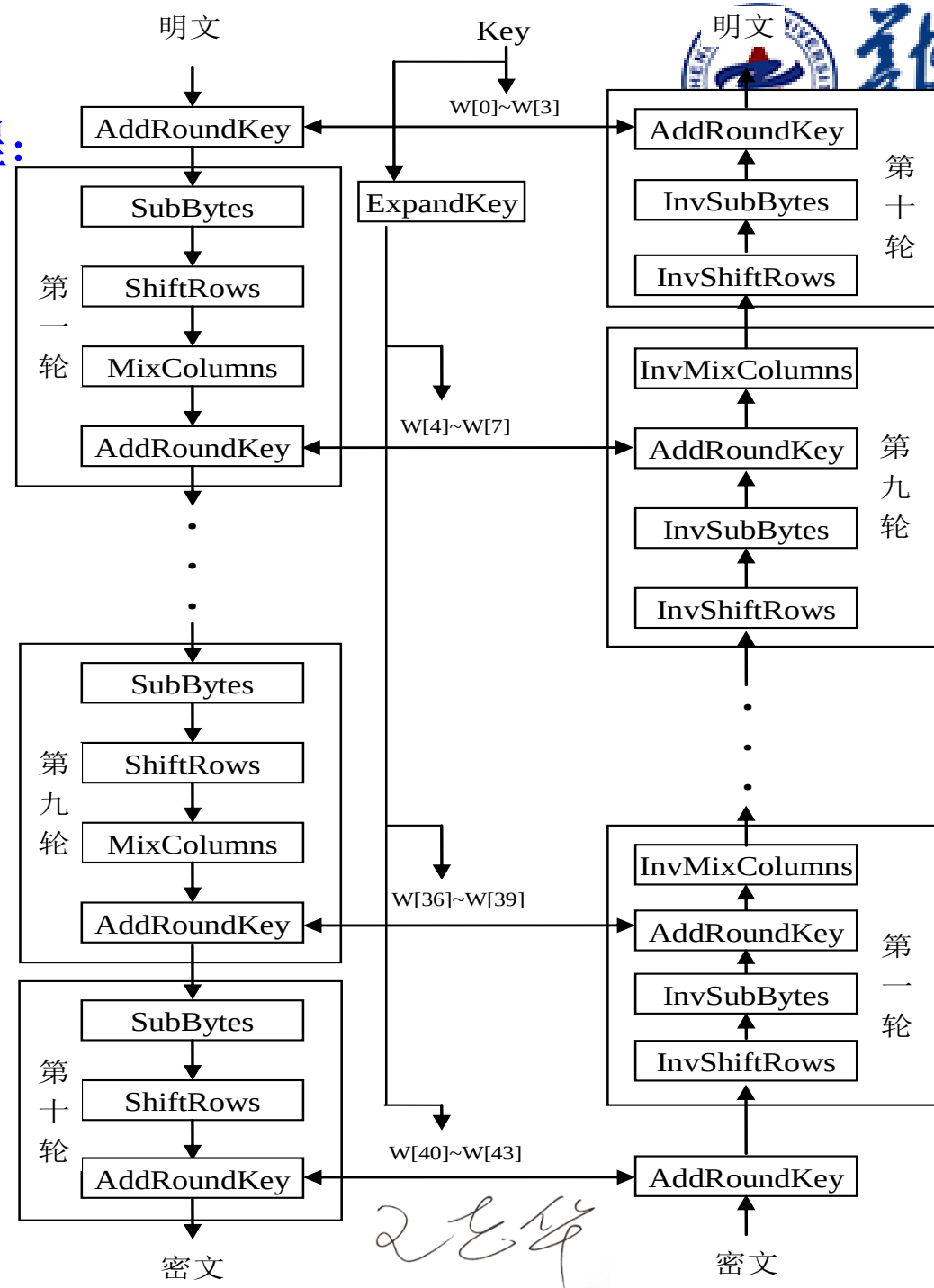


## 5、AES基本概念

- **Rijndael**也是一个迭代分组密码，其中明文分组长和密钥长度都可变，可以独立地指定为128比特、192比特、256比特。

2022/3/19

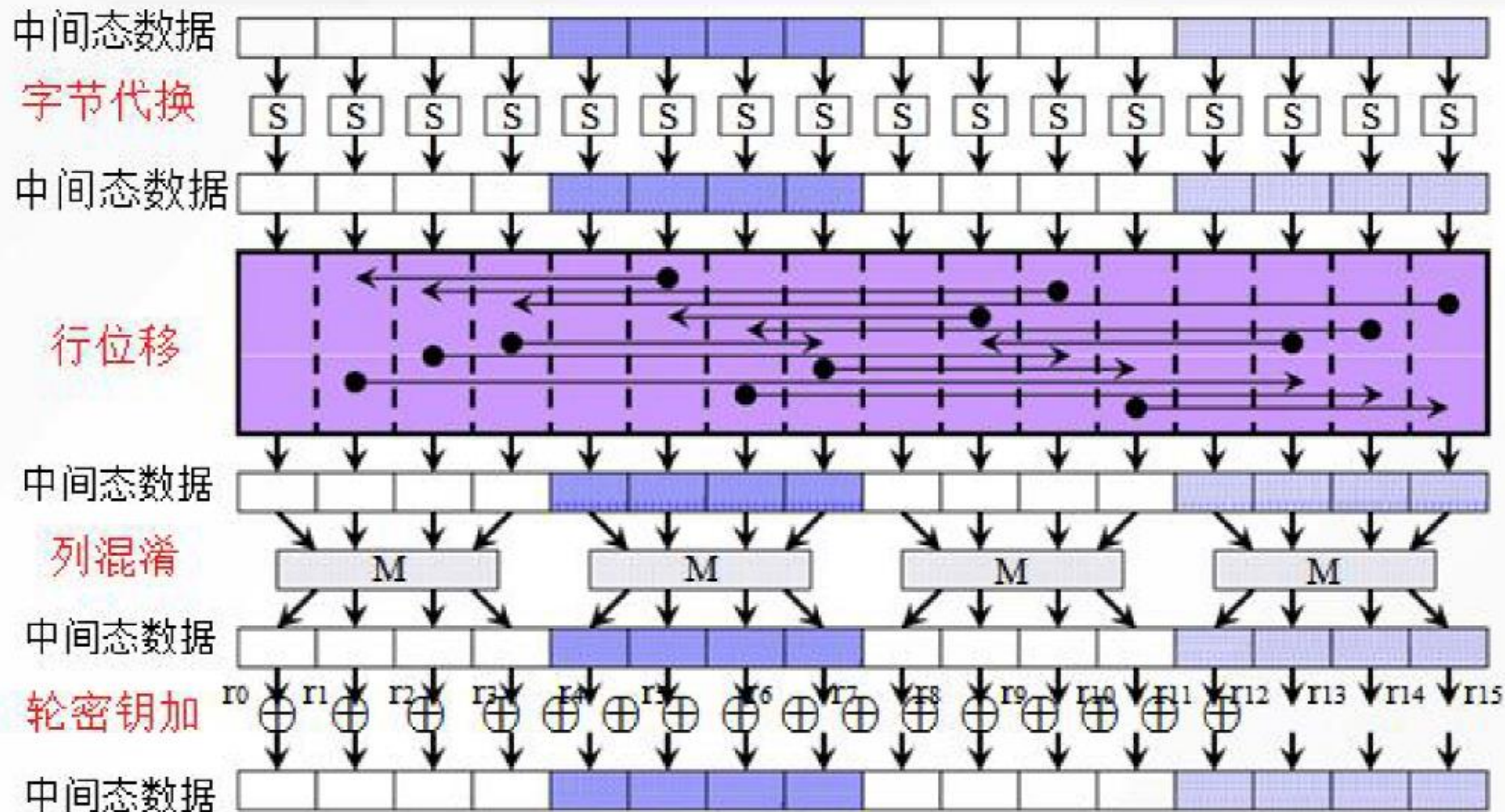
# AES算法过程:



2022/3/19



## AES一轮流程:



2024



## AES算法基本参数:

☞  $N_b$ : 明文分组长度标志, 即每个分组分为  $N_b$  个4字节 (32比特) 的字, 分组长度为128、192、256时,  $N_b$  分别取4、6、8。

☞  $N_k$ : 密钥分组长度标志, 即将密钥分成  $N_k$  个4字节的字, 同样,  $N_k$  也可取4、6、8。

☞  $N_r$ : 加密轮数, 或称迭代层数。  $N_r$  由  $N_b$  和  $N_k$  按下表决定:

| $N_r$   | $N_b=4$ | $N_b=6$ | $N_b=8$ |
|---------|---------|---------|---------|
| $N_k=4$ | 10      | 12      | 14      |
| $N_k=6$ | 12      | 12      | 14      |
| $N_k=8$ | 14      | 14      | 14      |

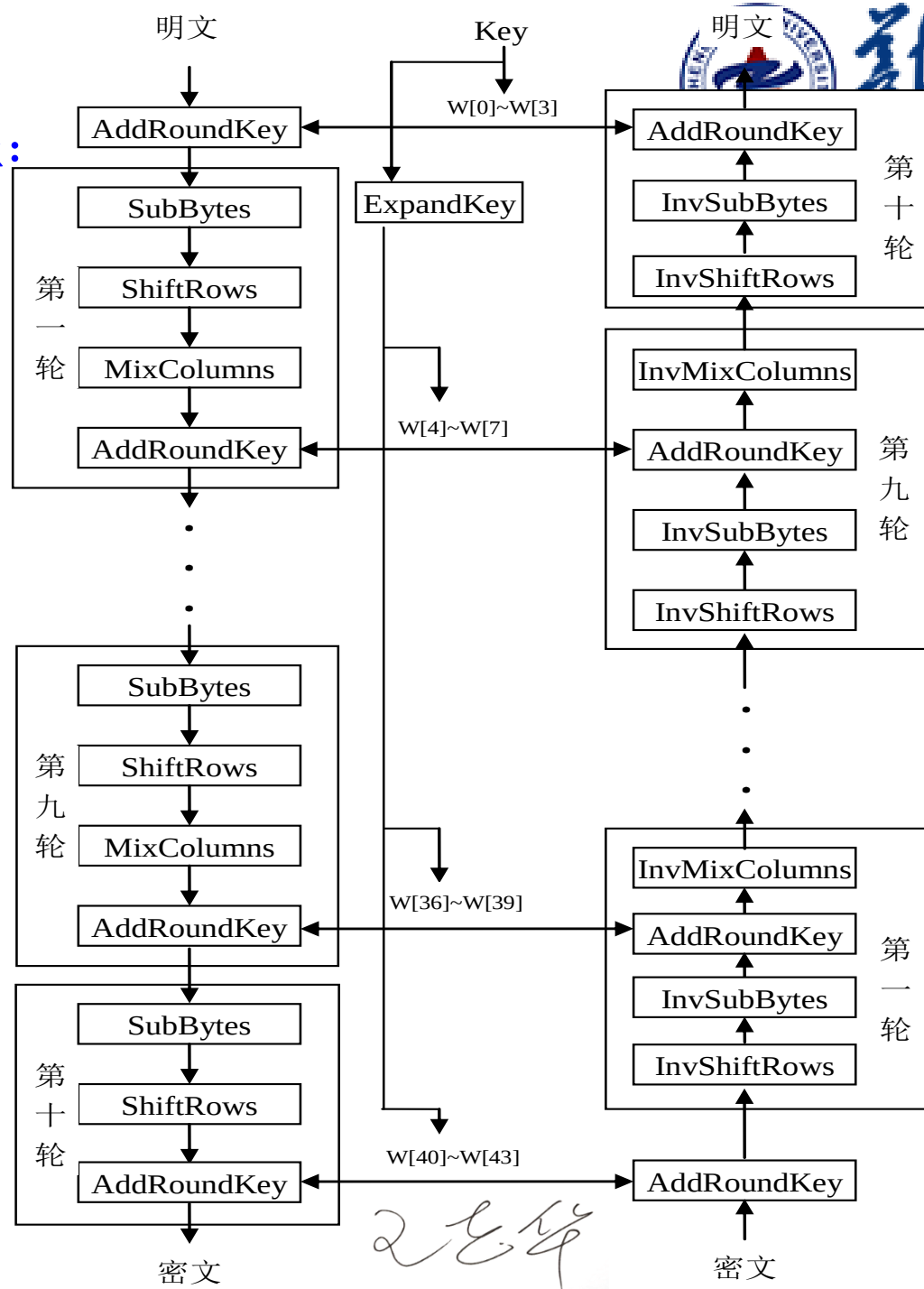
2022/3/19

## AES算法过程:

每一轮都要进行:

- 1.轮密钥混合
- 2.代换
- 3.置换
- 4.线性变换

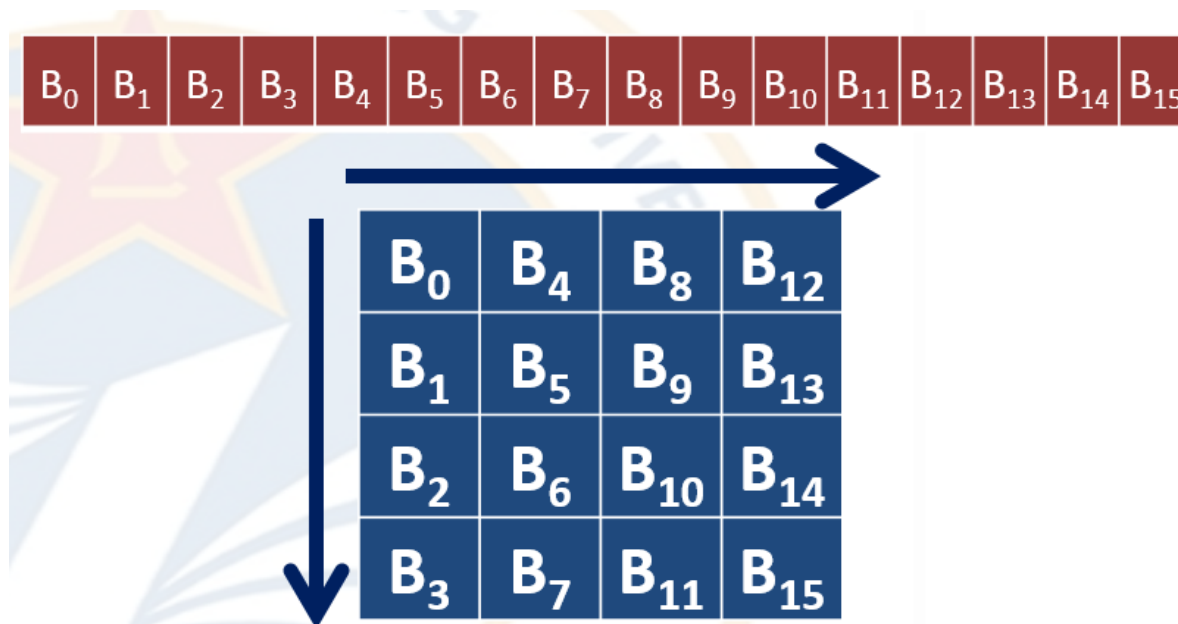
其中，包括“白化”过程。



王立华

## AES的状态:

- 类似于明文分组和密文分组，算法的中间结果也须分组，称**算法中间结果的分组**为**状态**，所有的操作都在状态上进行。
- 状态可以用以字节为元素的**矩阵阵列**表示，该阵列有4行，列数记为 $N_b$ ， $N_b$ 等于分组长度除以32。





## AES的种子密钥:

- 种子密钥类似地用一个以字节为元素的矩阵阵列表示，该阵列有4行，列数记为 $N_k$ ， $N_k$ 等于分组长度除以32。
- 由种子密钥可以计算各层的子**密钥** $w[i]$ 。

王华



## AES的状态和种子密钥的矩阵阵列表示:

表3-8是 $N_b=6$ 的状态和 $N_k=4$ 的种子密钥的矩阵阵列表示

表3-8  $N_b=6$ 的状态和 $N_k=4$ 的种子密钥

|          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|
| $a_{00}$ | $a_{01}$ | $a_{02}$ | $a_{03}$ | $a_{04}$ | $a_{05}$ |
| $a_{10}$ | $a_{11}$ | $a_{12}$ | $a_{13}$ | $a_{14}$ | $a_{15}$ |
| $a_{20}$ | $a_{21}$ | $a_{22}$ | $a_{23}$ | $a_{24}$ | $a_{25}$ |
| $a_{30}$ | $a_{31}$ | $a_{32}$ | $a_{33}$ | $a_{34}$ | $a_{35}$ |

|          |          |          |          |
|----------|----------|----------|----------|
| $k_{00}$ | $k_{01}$ | $k_{02}$ | $k_{03}$ |
| $k_{10}$ | $k_{11}$ | $k_{12}$ | $k_{13}$ |
| $k_{20}$ | $k_{21}$ | $k_{22}$ | $k_{23}$ |
| $k_{30}$ | $k_{31}$ | $k_{32}$ | $k_{33}$ |

2022/3/19





## AES的状态和种子密钥的一维数组表示:

- 也可将这些分组当作一维数组，其每一元素是上述阵列表示中的**4字节元素构成的列向量**，数组长度可为4、6、8，数组元素下标的范围分别是0~3、0~5和0~7。
- 4字节元素构成的列向量有时也称为字。

王卫华



## AES的状态和种子密钥的一维数组表示:

表3-8是 $N_b=6$ 的状态和 $N_k=4$ 的种子密钥的矩阵阵列表示

表 3-8  $N_b=6$  的状态和  $N_k=4$  的种子密钥

|          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|
| $a_{00}$ | $a_{01}$ | $a_{02}$ | $a_{03}$ | $a_{04}$ | $a_{05}$ |
| $a_{10}$ | $a_{11}$ | $a_{12}$ | $a_{13}$ | $a_{14}$ | $a_{15}$ |
| $a_{20}$ | $a_{21}$ | $a_{22}$ | $a_{23}$ | $a_{24}$ | $a_{25}$ |
| $a_{30}$ | $a_{31}$ | $a_{32}$ | $a_{33}$ | $a_{34}$ | $a_{35}$ |

|          |          |          |          |
|----------|----------|----------|----------|
| $k_{00}$ | $k_{01}$ | $k_{02}$ | $k_{03}$ |
| $k_{10}$ | $k_{11}$ | $k_{12}$ | $k_{13}$ |
| $k_{20}$ | $k_{21}$ | $k_{22}$ | $k_{23}$ |
| $k_{30}$ | $k_{31}$ | $k_{32}$ | $k_{33}$ |

按此顺序放入和读出

按此顺序放入

2020



## AES的输入明文和输出密文表示:

- 算法的输入和输出被看成是由**8比特的字节构成的一维数组**，其元素下标的范围是 $0 \sim (4N_b - 1)$ ，因此输入和输出以字节为单位的分组长度分别是16、24和32，其元素下标的范围分别是 $0 \sim 15$ 、 $0 \sim 23$ 和 $0 \sim 31$ 。
- 如果将明文  $a_0 a_1 a_2 \dots a_{15}$  （以128分组为例）按字节写成矩阵（State），其中每个元素是一字节：

|       |       |          |          |
|-------|-------|----------|----------|
| $a_0$ | $a_4$ | $a_8$    | $a_{12}$ |
| $a_1$ | $a_5$ | $a_9$    | $a_{13}$ |
| $a_2$ | $a_6$ | $a_{10}$ | $a_{14}$ |
| $a_3$ | $a_7$ | $a_{11}$ | $a_{15}$ |

2022



## AES的种子密钥表示:

- 输入的种子密钥也看成是由8比特字节构成的一维数组，其元素下标的范围是 $0 \sim (4N_k - 1)$ ，因此种子密钥以字节为单位的分组长度也分别是16、24和32，其元素下标的范围分别是 $0 \sim 15$ 、 $0 \sim 23$ 和 $0 \sim 31$ 。

王华



## AES的状态阵列和种子密钥阵列:

- 算法的输入（包括最初的明文输入和中间过程的轮输入）以字节为单位按 $a_{00}a_{10}a_{20}a_{30}a_{01}a_{11}a_{21}a_{31}\cdots$ 的顺序放置到**状态阵列**中。同理，种子密钥以字节为单位按 $k_{00}k_{10}k_{20}k_{30}k_{01}k_{11}k_{21}k_{31}\cdots$ 的顺序放置到**种子密钥阵列**中。
- 算法的输出（包括中间过程的轮输出和最后的密文输出），也是以字节为单位按相同的顺序从**状态阵列**中取出。

王华



## AES的状态阵列和元素位置之间的关系:

- 若输入（或输出）分组中第 $n$ 个元素对应于状态阵列的第 $(i,j)$ 位置上的元素，则 $n$ 和 $(i,j)$ 有以下关系：

$$i = n \bmod 4; j = \lfloor n/4 \rfloor; n = i + 4j$$

2022



## AES基于状态矩阵的转换过程:

算法的运行过程是将需要加密的分组从一个状态转换为另一个状态，最后该数组被复制到输出矩阵中

如对于**128**位分组，假设加密和解密的初始阶段将输入字节数组为复制到如图**3.21**所示的状态（**State**）矩阵中。加密或解密的运算都在该状态矩阵上进行，最后的结果将被复制到输出字节数组。

2022/3/19



## AES基于状态矩阵的转换过程:

输入字节

状态矩阵

输出字节

|        |        |           |           |
|--------|--------|-----------|-----------|
| $in_0$ | $in_4$ | $in_8$    | $in_{12}$ |
| $in_1$ | $in_5$ | $in_9$    | $in_{13}$ |
| $in_2$ | $in_6$ | $in_{10}$ | $in_{14}$ |
| $in_3$ | $in_7$ | $in_{11}$ | $in_{15}$ |



|           |           |           |           |
|-----------|-----------|-----------|-----------|
| $s_{0,0}$ | $s_{0,1}$ | $s_{0,2}$ | $s_{0,3}$ |
| $s_{1,0}$ | $s_{1,1}$ | $s_{1,2}$ | $s_{1,3}$ |
| $s_{2,0}$ | $s_{2,1}$ | $s_{2,2}$ | $s_{2,3}$ |
| $s_{3,0}$ | $s_{3,1}$ | $s_{3,2}$ | $s_{3,3}$ |



|         |         |            |            |
|---------|---------|------------|------------|
| $out_0$ | $out_4$ | $out_8$    | $out_{12}$ |
| $out_1$ | $out_5$ | $out_9$    | $out_{13}$ |
| $out_2$ | $out_6$ | $out_{10}$ | $out_{14}$ |
| $out_3$ | $out_7$ | $out_{11}$ | $out_{15}$ |

状态矩阵、输入和输出

- 状态矩阵中每一列的四个字节可以看作一个32-bit字, 用行号 $r$ 作为每一个字中四个字节的索引
- 状态可以看作32-bit字(列), 的一维数组, 用列号 $c$ 表示该数组的索引

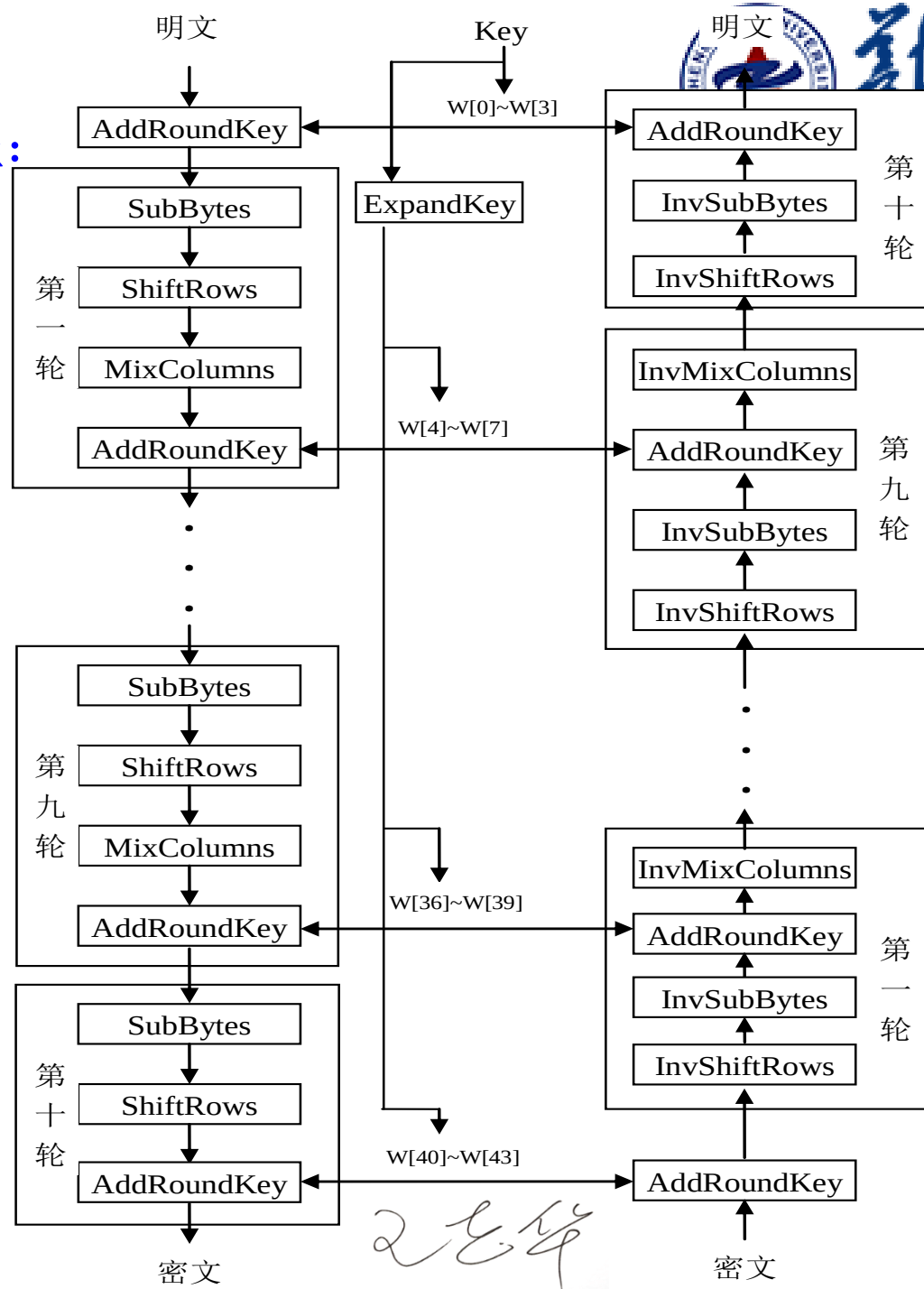


## AES算法过程:

每一轮都要进行:

- 1.轮密钥混合
- 2.代换
- 3.置换
- 4.线性变换

其中，包括“白化”过程。



王立华



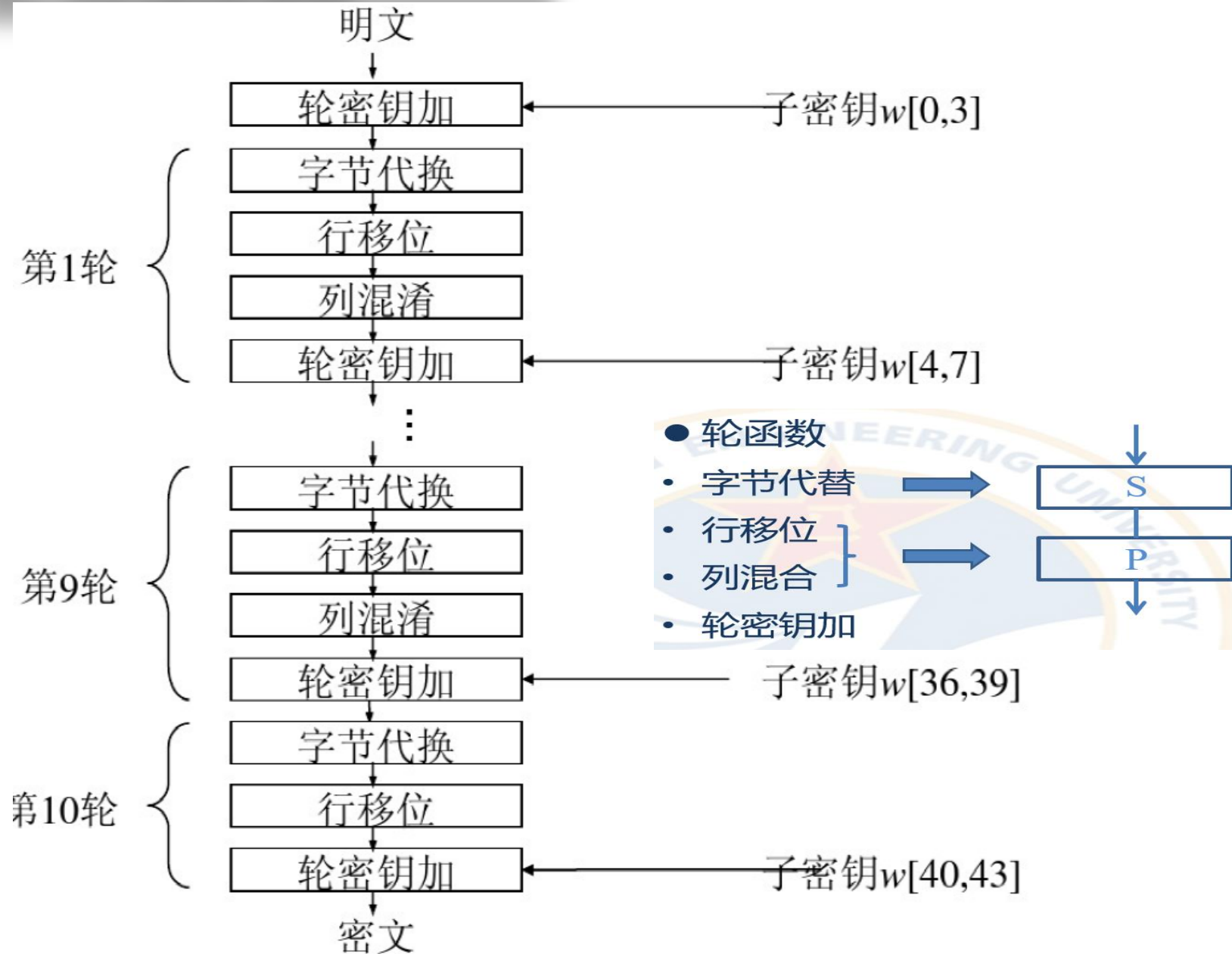
## 6、AES的轮函数

- 当分组长度和密钥长度均为128位时，AES共迭代10轮，需要11个子密钥。
- 前面9轮完全相同，每轮包括4阶段，分别是字节代换（SubBytes）、行移位（Shift Rows）、列混淆（Mix Columns）和轮密钥加（Add Round Key）
- 最后一轮只3个阶段，缺少列混淆。

2022/3/19

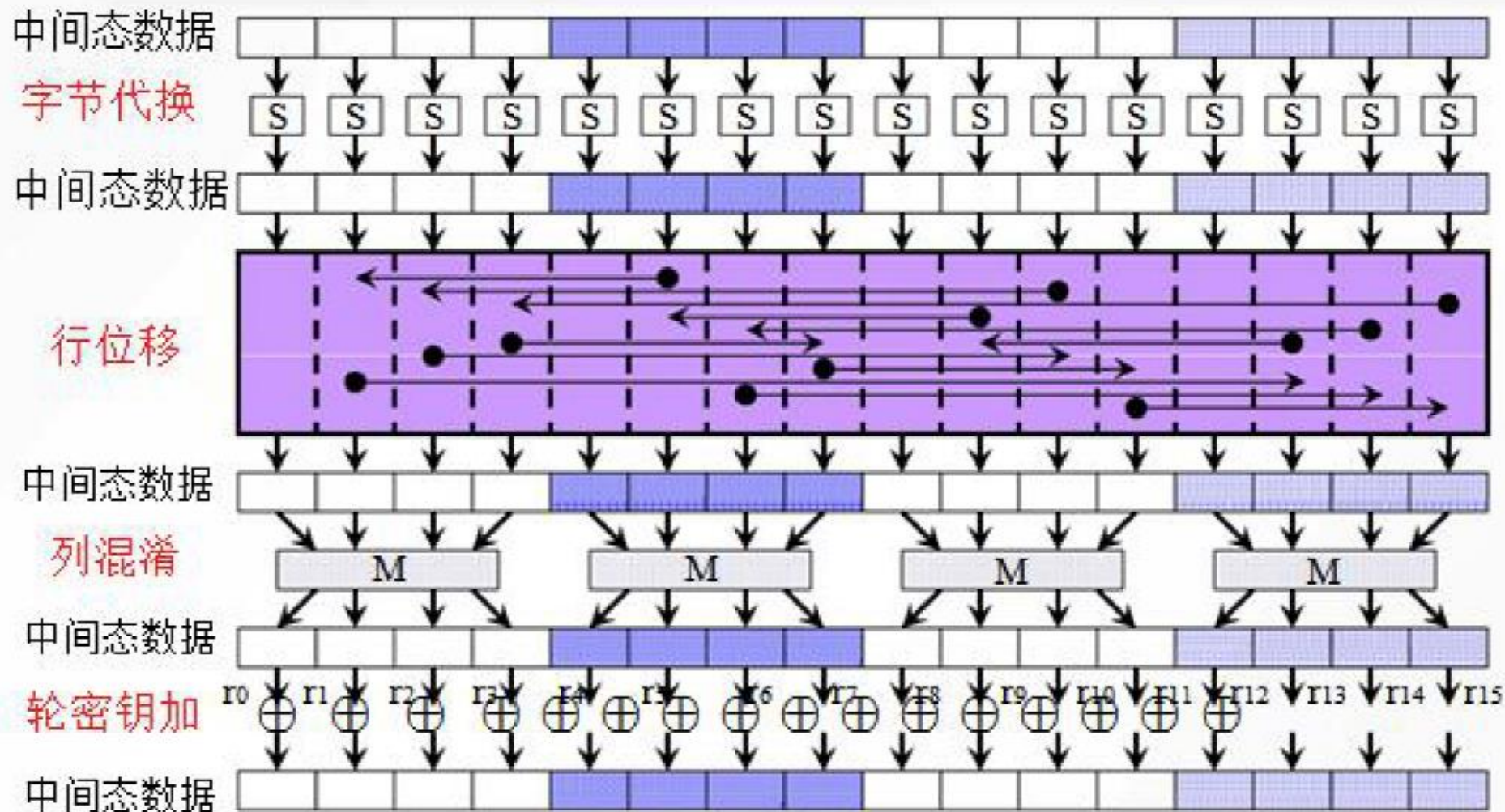


## 6、AES的轮函数





## AES一轮流程:



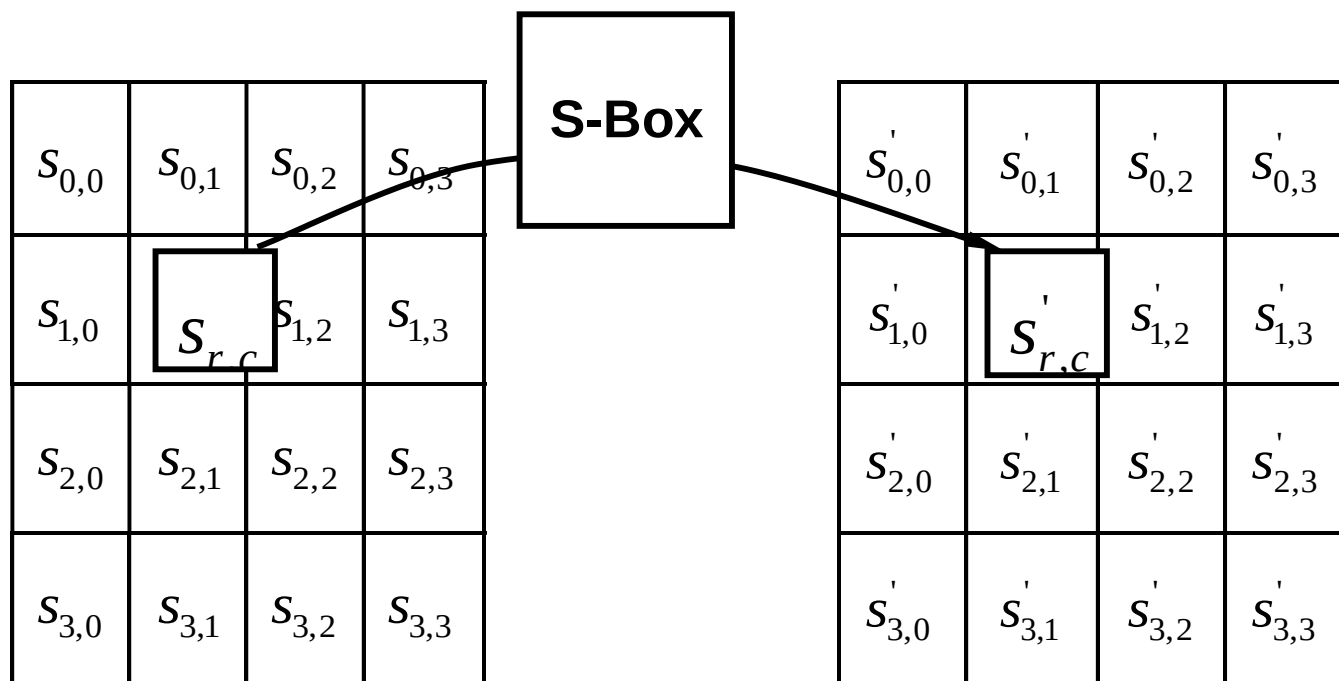
2024



## AES轮函数：字节代换

### 字节代换 (ByteSub)

- 字节代换是非线性变换，独立地对状态的每个字节（8个比特）进行。
- 字节变换由多个S-盒构成。



2024



# AES轮函数：S盒(x和y构成一个字节)



郑州大学

|   |   | y  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
|   |   | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | a  | b  | c  | d  | e  | f  |
| x | 0 | 63 | 7c | 77 | 7b | f2 | 6b | 6f | c5 | 30 | 01 | 67 | 2b | fe | d7 | ab | 76 |
|   | 1 | ca | 82 | c9 | 7d | fa | 59 | 47 | f0 | ad | d4 | a2 | af | 9c | a4 | 72 | c0 |
|   | 2 | b7 | fd | 93 | 26 | 36 | 3f | f7 | cc | 34 | a5 | e5 | f1 | 71 | d8 | 31 | 15 |
|   | 3 | 04 | c7 | 23 | c3 | 18 | 96 | 05 | 9a | 07 | 12 | 80 | e2 | eb | 27 | b2 | 75 |
|   | 4 | 09 | 83 | 2c | 1a | 1b | 6e | 5a | a0 | 52 | 3b | d6 | b3 | 29 | e3 | 2f | 84 |
|   | 5 | 53 | d1 | 00 | ed | 20 | fc | b1 | 5b | 6a | cb | be | 39 | 4a | 4c | 58 | cf |
|   | 6 | d0 | ef | aa | fb | 43 | 4d | 33 | 85 | 45 | f9 | 02 | 7f | 50 | 3c | 9f | a8 |
|   | 7 | 51 | a3 | 40 | 8f | 92 | 9d | 38 | f5 | bc | b6 | da | 21 | 10 | ff | f3 | d2 |
|   | 8 | cd | 0c | 13 | ec | 5f | 97 | 44 | 17 | c4 | a7 | 7e | 3d | 64 | 5d | 19 | 73 |
|   | 9 | 60 | 81 | 4f | dc | 22 | 2a | 90 | 88 | 46 | ee | b8 | 14 | de | 5e | 0b | db |
|   | a | e0 | 32 | 3a | 0a | 49 | 06 | 24 | 5c | c2 | d3 | ac | 62 | 91 | 95 | e4 | 79 |
|   | b | e7 | c8 | 37 | 6d | 8d | d5 | 4e | a9 | 6c | 56 | f4 | ea | 65 | 7a | ae | 08 |
|   | c | ba | 78 | 25 | 2e | 1c | a6 | b4 | c6 | e8 | dd | 74 | 1f | 4b | bd | 8b | 8a |
|   | d | 70 | 3e | B5 | 66 | 48 | 03 | f6 | 0e | 61 | 35 | 57 | b9 | 86 | c1 | 1d | 9e |
|   | e | e1 | f8 | 98 | 11 | 69 | d9 | 8e | 94 | 9b | 1e | 87 | e9 | ce | 55 | 28 | df |
|   | f | 8c | a1 | 89 | 0d | bf | e6 | 42 | 68 | 41 | 99 | 2d | 0f | B0 | 54 | bb | 16 |



## AES轮函数：字节代换

### ● 字节代换 (ByteSub)

- AES的S盒能够被**代数地定义**，而不像DES的S盒那样是明显的“随机”代换；
- AES的S盒的代数公式包含有限域上的操作。

王华



## AES轮函数：S盒中的位运算

设S盒中的8位是 $a_7a_6a_5a_4a_3a_2a_1a_0$ ，它在有限域中的逆是 $b_7b_6b_5b_4b_3b_2b_1b_0$ ，则

$$b'_i = b_i \oplus b_{(i+4) \bmod 8} \oplus b_{(i+5) \bmod 8} \\ \oplus b_{(i+6) \bmod 8} \oplus b_{(i+7) \bmod 8} \oplus c_i$$

将状态中的一个字节记作 $S_{r,c}$ ，

仿射变换记为 $f$ ，则S-盒可表示为：

$$S'_{r,c} = f(S^{-1}_{r,c})$$

吴志华





## AES轮函数：字节代换

### ● 字节代换 (ByteSub)

➤ 代换表（即S-盒）是可逆的，由以下两个变换的合成得到。

- ① 首先，将字节看作  $GF(2^8)$  上的元素，映射到自己的**乘法逆元**，‘00’映射到自己
- ② 其次，对字节做如下的（GF(2)上的，可逆的）**字节变换**

王华



## AES轮函数：字节代换

### ● 字节代换 (ByteSub)

整个变换做成256字节的代换表，运算时查表即可

$$\begin{pmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \\ y_6 \\ y_7 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}$$

2022



## AES轮函数：S盒代换示例

|    |    |    |    |
|----|----|----|----|
| EA | 04 | 65 | 85 |
| 83 | 45 | 5D | 96 |
| 5C | 33 | 98 | B0 |
| F0 | 2D | AD | C5 |

S盒  
→

|    |    |    |    |
|----|----|----|----|
| 87 | F2 | 4D | 97 |
| EC | 6E | 4C | 90 |
| 4A | C3 | 46 | E7 |
| 8C | D8 | 95 | A6 |



## AES轮函数：S盒

- 如用{95}作为输入，可以计算出{95}在 $GF(2^8)$ 上的逆为{8A}，用二进制表示为10001010，代入上述变换有

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \oplus \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} \oplus \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 1 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

- 结果为00101010，用十六进制表示为{2A}，与前面查表所得结果一样

✓✓✓

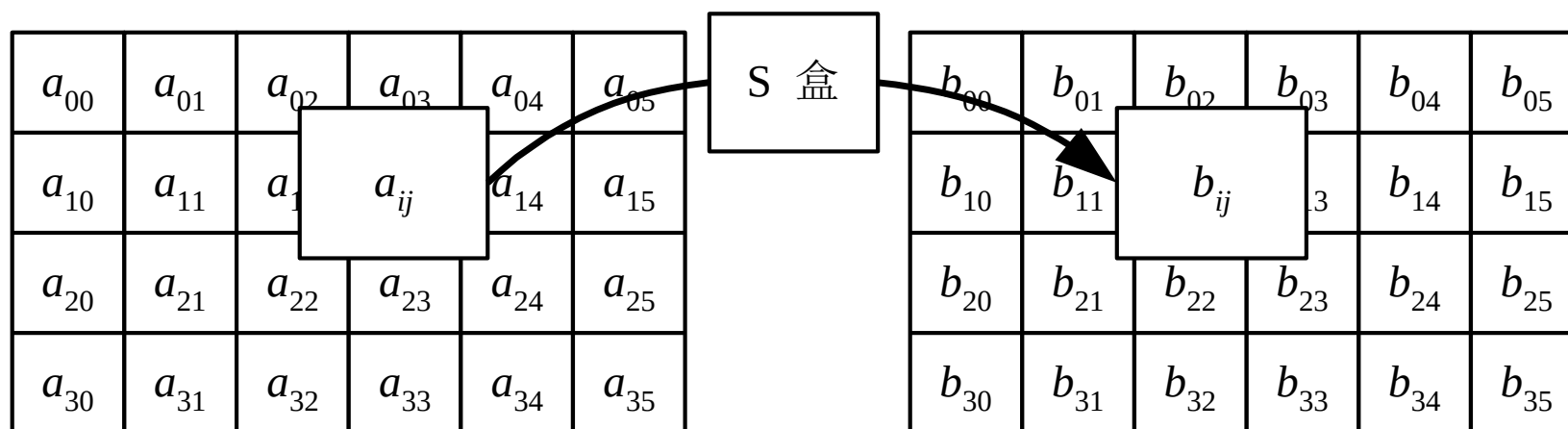


## AES轮函数：字节代换

### ● 字节代换 (ByteSub)

➤ 上述S-盒对状态的所有字节所做的变换记为：

ByteSub (State)



字节代换示意图

2022/3/19



## AES轮函数：S盒评价

- S盒被设计成能防止已有的各种密码分析攻击。Rijndael的开发者优先寻求在输入位和输出位之间几乎没有相关性的设计，且输出值不能通过利用一个简单的数学函数变换输入值所得。另外，在代码变换中所选择的常量使得在S盒中没有不动点 $[S_{\text{盒}}(a)=a]$ ，也没有“反不动点” $[S_{\text{盒}}(a)=\text{逆}a]$ 。
- 当然，S盒必须是可逆的，即 $\text{逆}S_{\text{盒}}[S_{\text{盒}}(a)]=a$ 。然而， $S_{\text{盒}}(a)=\text{逆}S_{\text{盒}}(a)$ 不成立，在这个意义上S盒不是自逆的。例如， $S_{\text{盒}}(\{95\})=\{2A\}$ ，但 $\text{逆}S_{\text{盒}}(\{95\})=\{AD\}$ 。

2022/3/19



## AES轮函数：字节代换

### ● 字节代换 (ByteSub)

➤ ByteSub的逆变换由代换表的逆表做字节代换，  
可通过如下两步实现：

- ① 首先进行字节变换的逆变换
- ② 再求每一字节在  $GF(2^8)$  上逆元。

王华



# AES轮函数：逆S盒

|   |   | y  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
|   |   | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | a  | b  | c  | d  | e  | f  |
| x | 0 | 52 | 09 | 6a | d5 | 30 | 36 | a5 | 38 | bf | 40 | a3 | 9e | 81 | f3 | d7 | fb |
|   | 1 | 7c | e3 | 39 | 82 | 9b | 2f | ff | 87 | 34 | 8e | 43 | 44 | c4 | de | e9 | cb |
|   | 2 | 54 | 7b | 94 | 32 | a6 | c2 | 23 | 3d | ee | 4c | 95 | 0b | 42 | fa | c3 | 4e |
|   | 3 | 08 | 2e | A1 | 66 | 28 | d9 | 24 | b2 | 76 | 5b | a2 | 49 | 6d | 8b | d1 | 25 |
|   | 4 | 72 | f8 | f6 | 64 | 86 | 68 | 98 | 16 | d4 | a4 | 5c | cc | 5d | 65 | B6 | 92 |
|   | 5 | 6c | 70 | 48 | 50 | fd | ed | b9 | da | 5e | 15 | 46 | 57 | a7 | 8d | 9d | 84 |
|   | 6 | 90 | d8 | ab | 00 | 8c | bc | d3 | 0a | f7 | e4 | 58 | 05 | b8 | b3 | 45 | 06 |
|   | 7 | d0 | 2c | 1e | 8f | ca | 3f | 0f | 02 | c1 | af | bd | 03 | 01 | 13 | 8a | 6b |
|   | 8 | 3a | 91 | 11 | 41 | 4f | 67 | dc | ea | 97 | f2 | cf | ce | f0 | b4 | e6 | 73 |
|   | 9 | 96 | ac | 74 | 22 | e7 | ad | 35 | 85 | e2 | f9 | 37 | e8 | 1c | 75 | df | 6e |
|   | a | 47 | f1 | 1a | 71 | 1d | 29 | c5 | 89 | 6f | b7 | 62 | 0e | aa | 18 | be | 1b |
|   | b | fc | 56 | 3e | 4b | c6 | d2 | 79 | 20 | 9a | db | c0 | fe | 78 | cd | 5a | f4 |
|   | c | 1f | dd | a8 | 33 | 88 | 07 | c7 | 31 | b1 | 12 | 10 | 59 | 27 | 80 | ec | 5f |
|   | d | 60 | 51 | 7f | a9 | 19 | b5 | 4a | 0d | 2d | e5 | 7a | 9f | 93 | c9 | 9c | ef |
|   | e | a0 | e0 | 3b | 4d | ae | 2a | f5 | b0 | c8 | eb | bb | 3c | 83 | 53 | 99 | 61 |
|   | f | 17 | 2b | 04 | 7e | ba | 77 | d6 | 26 | e1 | 69 | 14 | 63 | 55 | 21 | 0c | 7d |





## AES轮函数：字节代换

FieldInv 表示求一个域元素的乘法逆；  
BinaryToField 把一个字节变换成一个域元素；  
FieldToBinary 进行相反的变换。

这个变换以一种明显的方式进行：域元素

$$\sum_{i=0}^7 a_i x^i$$

对应于字节

$$a_7 a_6 a_5 a_4 a_3 a_2 a_1 a_0$$

**算法 3.4** SubBytes( $a_7 a_6 a_5 a_4 a_3 a_2 a_1 a_0$ )

**external** FieldInv, BinaryToField, FieldToBinary

$z \leftarrow \text{BinaryToField}(a_7 a_6 a_5 a_4 a_3 a_2 a_1 a_0)$

**if**  $z \neq 0$

**then**  $z \leftarrow \text{FieldInv}(z)$

$(a_7 a_6 a_5 a_4 a_3 a_2 a_1 a_0) \leftarrow \text{FieldToBinary}(z)$

$(c_7 c_6 c_5 c_4 c_3 c_2 c_1 c_0) \leftarrow (01100011)$

**comment:** 在下面的循环中, 所有下标都要经过模 8 约简

**for**  $i \leftarrow 0$  **to** 7

**do**  $b_i \leftarrow (a_i + a_{i+4} + a_{i+5} + a_{i+6} + a_{i+7} + c_i) \bmod 2$

**return** ( $b_7 b_6 b_5 b_4 b_3 b_2 b_1 b_0$ )

2022/3/19



## AES轮函数：行移位

### ● 行移位 (ShiftRow)

➤ 行移位是将状态阵列的各行（分组为 4、6、8）进行循环移位，以字节为单位，不同状态行的位移量不同。

- ① 第0行不移动，
- ② 第1行循环左移 $C_1$ 个字节，
- ③ 第2行循环左移 $C_2$ 个字节，
- ④ 第3行循环左移 $C_3$ 个字节。

➤ 位移量 $C_1$ 、 $C_2$ 、 $C_3$ 的取值与 $N_b$ 有关：  
对应于不同分组长度的位移量

| $N_b$ | $C_1$ | $C_2$ | $C_3$ |
|-------|-------|-------|-------|
| 4     | 1     | 2     | 3     |
| 6     | 1     | 2     | 3     |
| 8     | 1     | 3     | 4     |

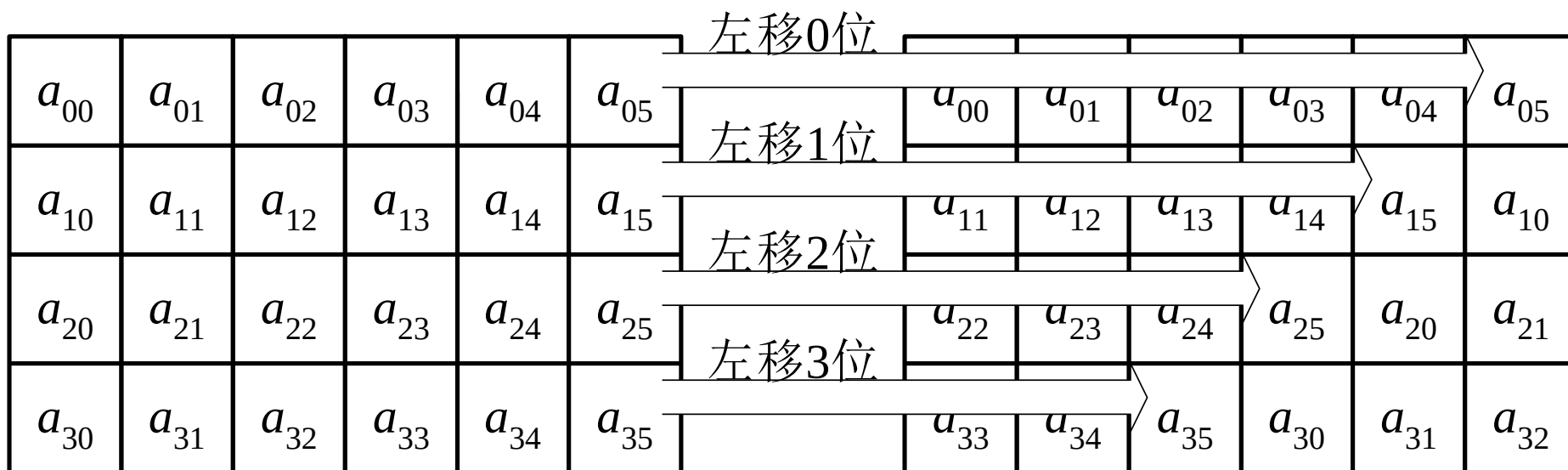


## AES轮函数：行移位

### 行移位 (ShiftRow)

- 按指定的位移量对状态的行进行的行移位运算记为：

ShiftRow (State): 分组为6的行变换过程



行移位示意图

王立华



## AES轮函数：行移位

### ● 行移位 (ShiftRow)

➤ 行变换规则用公式表示如下：

$$S'_{r,c} = S_{r,(c+shift(r,Nb)) \bmod Nb}$$

$$shift(1,4) = 1$$

$$shift(2,4) = 2$$

$$shift(3,4) = 3$$

2024

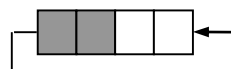
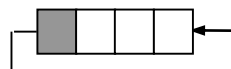
## AES轮函数：行移位

### 行移位 (ShiftRow)

➤ 分组为4的行变换过程：

$S$

|           |           |           |           |
|-----------|-----------|-----------|-----------|
| $S_{0,0}$ | $S_{0,1}$ | $S_{0,2}$ | $S_{0,3}$ |
| $S_{1,0}$ | $S_{1,1}$ | $S_{1,2}$ | $S_{1,3}$ |
| $S_{2,0}$ | $S_{2,1}$ | $S_{2,2}$ | $S_{2,3}$ |
| $S_{3,0}$ | $S_{3,1}$ | $S_{3,2}$ | $S_{3,3}$ |



$S'$

|           |           |           |           |
|-----------|-----------|-----------|-----------|
| $S_{0,0}$ | $S_{0,1}$ | $S_{0,2}$ | $S_{0,3}$ |
| $S_{1,1}$ | $S_{1,2}$ | $S_{1,3}$ | $S_{1,0}$ |
| $S_{2,2}$ | $S_{2,3}$ | $S_{2,0}$ | $S_{2,1}$ |
| $S_{3,3}$ | $S_{3,0}$ | $S_{3,1}$ | $S_{3,2}$ |

2022/3/19



## AES轮函数：行移位示例

|    |    |    |    |
|----|----|----|----|
| 87 | F2 | 4D | 97 |
| EC | 6E | 4C | 90 |
| 4A | C3 | 46 | E7 |
| 8C | D8 | 95 | A6 |

行移位  
→

|           |           |           |           |
|-----------|-----------|-----------|-----------|
| 87        | <i>F2</i> | <i>4D</i> | <i>97</i> |
| <i>6E</i> | <i>4C</i> | <i>90</i> | <i>EC</i> |
| <i>46</i> | <i>E7</i> | <i>4A</i> | <i>C3</i> |
| <i>A6</i> | <i>8C</i> | <i>D8</i> | <i>95</i> |



## AES轮函数：行移位说明

- 由于中间态数据和算法的输入输出数据一样，也是由4列所组成的数组，在加密过程中，明文逐列被复制到中间态数据上，且后面的轮密钥也是逐列应用到中间态数据上，因此，行位移将某个字节从一列移到另一列中，这个变换确保了某列中的4字节被扩展到了4个不同的列。

2022



## AES轮函数：行移位

### ● 行移位 (ShiftRow)

- **ShiftRow的逆变换**是对状态阵列的后三列分别以位移量  $N_b - C_1$ 、 $N_b - C_2$ 、 $N_b - C_3$ 进行循环移位，使得第  $i$  行第  $j$  列的字节移位到  $(j + N_b - C_i) \bmod N_b$

2022/3/19

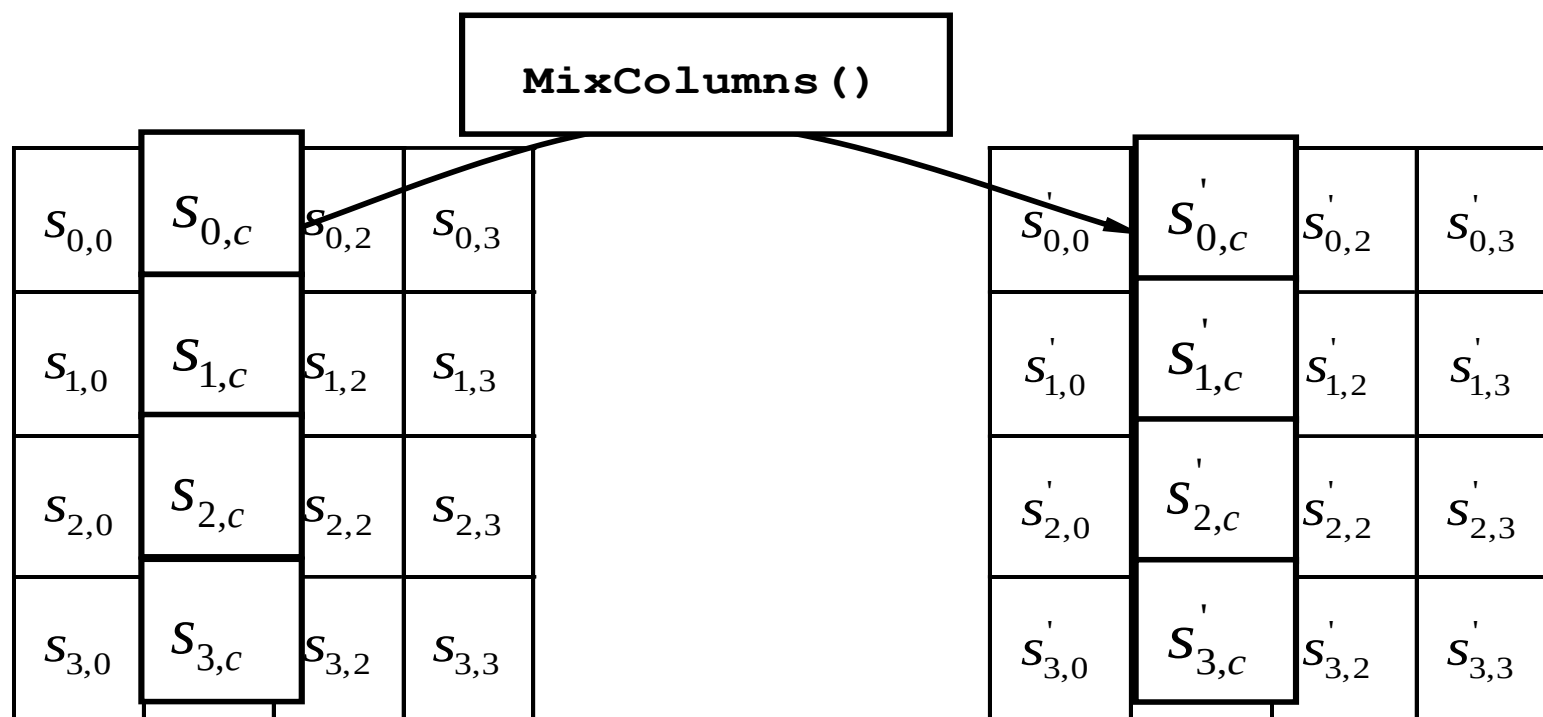




## AES轮函数：列混合

### ● 列混合 (MixColumn)

➤ 列变换就是对每一列作变换：



2022/3/19



## AES轮函数：列混合

### ● 列混合 (MixColumn)

- 在列混合变换中，将状态阵列的每个列视为  $GF(2^8)$  上的多项式，再与一个固定的多项式  $c(x)$  进行模  $x^4+1$  乘法。
- 要求， $c(x)$  是模  $x^4+1$  可逆的多项式，否则列混合变换就是不可逆的，因而会使不同的输入分组对应的输出分组可能相同。

2022/3/19



## AES轮函数：列混合

### ● 列混合 (MixColumn)

- Rijndael的设计者给出的 $c(x)$ 为（系数用16进制数表示）：

$$c(x) = '03'x^3 + '01'x^2 + '01'x + '02'$$

- $c(x)$ 是与 $x^4+1$ 互素的，因此是模 $x^4+1$ 可逆的。

**定理3-1** 系数在 $GF(2^8)$ 上的多项式  $a_3x^3 + a_2x^2 + a_1x + a_0$  是模 $x^4+1$ 可逆的，当且仅当**矩阵**

$$\begin{pmatrix} a_0 & a_3 & a_2 & a_1 \\ a_1 & a_0 & a_3 & a_2 \\ a_2 & a_1 & a_0 & a_3 \\ a_3 & a_2 & a_1 & a_0 \end{pmatrix}$$

在 $GF(2^8)$ 上可逆。

2022/3/19



## AES轮函数：列混合计算

### ● 列混合 (MixColumn)

- 列变换是对列作一个运算，得到一个新的列的过程，如上定义：

$$c(x) = '03'x^3 + '01'x^2 + '01'x + '02'$$

- 将4字节的字{03, 01, 01, 02}对应为 $c(x)$ ，State中的一个列对应到 $a(x)$ ，则新的列就是 $c(x) \otimes a(x)$ 所对应的4字节：

$$b(x) = c(x) \otimes s(x)$$

2024



## AES轮函数：列混合

### ● 列混合 (MixColumn)

- 列混合运算也可写为**矩阵乘法**。设 $b(x) = c(x) \otimes a(x)$ ，则：

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{pmatrix} = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{pmatrix}$$

- 这个运算需要做 $GF(2^8)$ 上的乘法，但由于所乘的因子是三个固定的元素02、03、01，所以这些乘法运算仍然是比较简单的。



## AES轮函数：列混合

### ● 列混合 (MixColumn)

➤ 展开表示为：

$$s'_{0,c} = (\{02\} \bullet s_{0,c}) \oplus (\{03\} \bullet s_{1,c}) \oplus s_{2,c} \oplus s_{3,c}$$

$$s'_{1,c} = s_{0,c} \oplus (\{02\} \bullet s_{1,c}) \oplus (\{03\} \bullet s_{2,c}) \oplus s_{3,c}$$

$$s'_{2,c} = s_{0,c} \oplus s_{1,c} \oplus (\{02\} \bullet s_{2,c}) \oplus (\{03\} \bullet s_{3,c})$$

$$s'_{3,c} = (\{03\} \bullet s_{0,c}) \oplus s_{1,c} \oplus s_{2,c} \oplus (\{02\} \bullet s_{3,c}).$$

2022



## AES轮函数：列混合

|           |           |           |           |
|-----------|-----------|-----------|-----------|
| $b_{0,0}$ | $b_{0,1}$ | $b_{0,2}$ | $b_{0,3}$ |
| $b_{1,1}$ | $b_{1,2}$ | $b_{1,3}$ | $b_{1,0}$ |
| $b_{2,2}$ | $b_{2,3}$ | $b_{2,0}$ | $b_{2,1}$ |
| $b_{3,3}$ | $b_{3,0}$ | $b_{3,1}$ | $b_{3,2}$ |

矩阵相乘

|           |           |           |           |
|-----------|-----------|-----------|-----------|
| $c_{0,0}$ | $c_{0,1}$ | $c_{0,2}$ | $c_{0,3}$ |
| $c_{1,1}$ | $c_{1,2}$ | $c_{1,3}$ | $c_{1,0}$ |
| $c_{2,2}$ | $c_{2,3}$ | $c_{2,0}$ | $c_{2,1}$ |
| $c_{3,3}$ | $c_{3,0}$ | $c_{3,1}$ | $c_{3,2}$ |

$$\begin{vmatrix} c_0 \\ c_1 \\ c_2 \\ c_3 \end{vmatrix} = \begin{vmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{vmatrix} \begin{vmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{vmatrix}$$

举例

$$\begin{vmatrix} 47 \\ 37 \\ 94 \\ ED \end{vmatrix} = \begin{vmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{vmatrix} \begin{vmatrix} 87 \\ 6E \\ 46 \\ A6 \end{vmatrix}$$

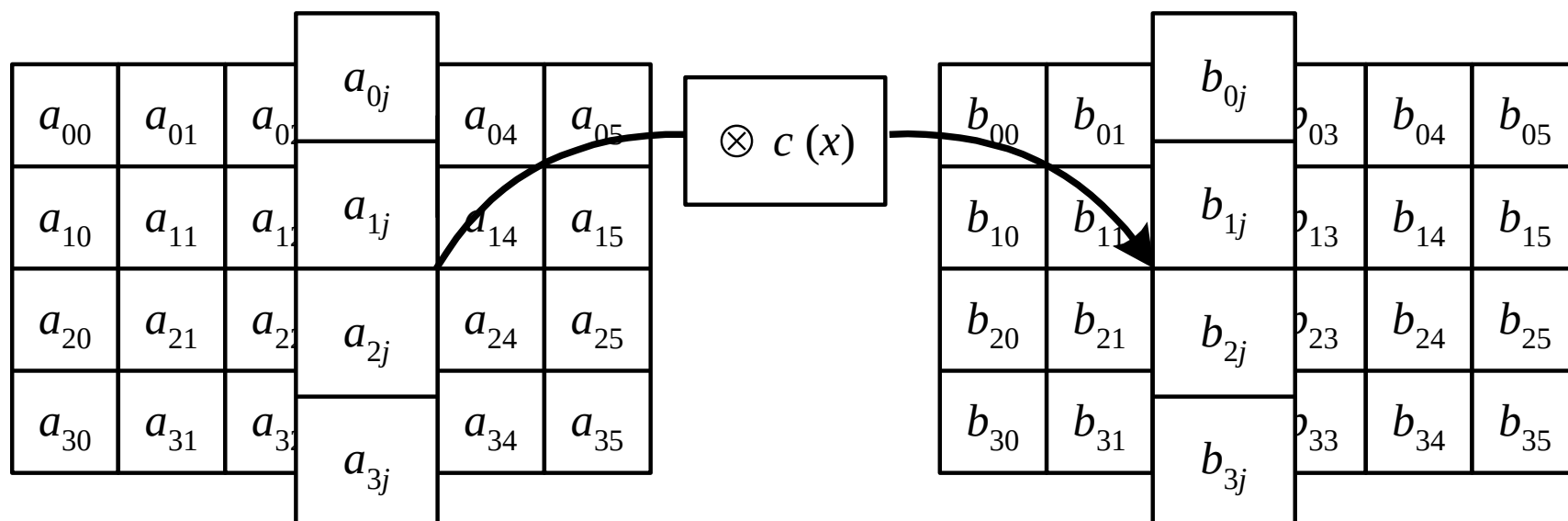
2022/3/19

## AES轮函数：列混合

### ● 列混合 (MixColumn)

➤ 对状态State的所有列所做的列混合运算记为：

MixColumn (State)



列混合运算示意图

2022/3/19





## AES轮函数：列混合

### ● 列混合 (MixColumn)

- 列混合运算的逆运算是类似的，即每列都用一个特定的多项式 $d(x)$ 相乘。 $d(x)$ 满足：

$$('03'x^3+'01'x^2+'01'x+'02') \otimes d(x) = '01'$$

- 因此，可得：

$$d(x) = '0B'x^3+'0D'x^2+'09'x+'0E'$$

2022/3/19



## AES轮函数：列混合

### ● 列混合 (MixColumn)

➤ 逆列混合可以写成矩阵乘法形式：

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_2 \end{pmatrix} = \begin{pmatrix} 0e & 0b & 0d & 09 \\ 09 & 0e & 0b & 0d \\ 0d & 09 & 0e & 0b \\ 0b & 0d & 09 & 0e \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{pmatrix}$$

2022



## AES轮函数：列混合

外部进程 `FieldMult` 以两个域元素为输入,并能计算它们在域中的 乘积  
域加法就是分量的模 2 加(即对应比特串的异或)。

### 算法 3.5 `MixColumn(c)`

**external** `FieldMult`, `BinaryToField`, `FieldToBinary`

**for**  $i \leftarrow 0$  **to** 3

**do**  $t_i \leftarrow \text{BinaryToField}(S_{i,c})$

$u_0 \leftarrow \text{FieldMult}(x, t_0) \oplus \text{FieldMult}(x + 1, t_1) \oplus t_2 \oplus t_3$

$u_1 \leftarrow \text{FieldMult}(x, t_1) \oplus \text{FieldMult}(x + 1, t_2) \oplus t_3 \oplus t_0$

$u_2 \leftarrow \text{FieldMult}(x, t_2) \oplus \text{FieldMult}(x + 1, t_3) \oplus t_0 \oplus t_1$

$u_3 \leftarrow \text{FieldMult}(x, t_3) \oplus \text{FieldMult}(x + 1, t_0) \oplus t_1 \oplus t_2$

**for**  $i \leftarrow 0$  **to** 3

**do**  $S_{i,c} \leftarrow \text{FieldToBinary}(u_i)$

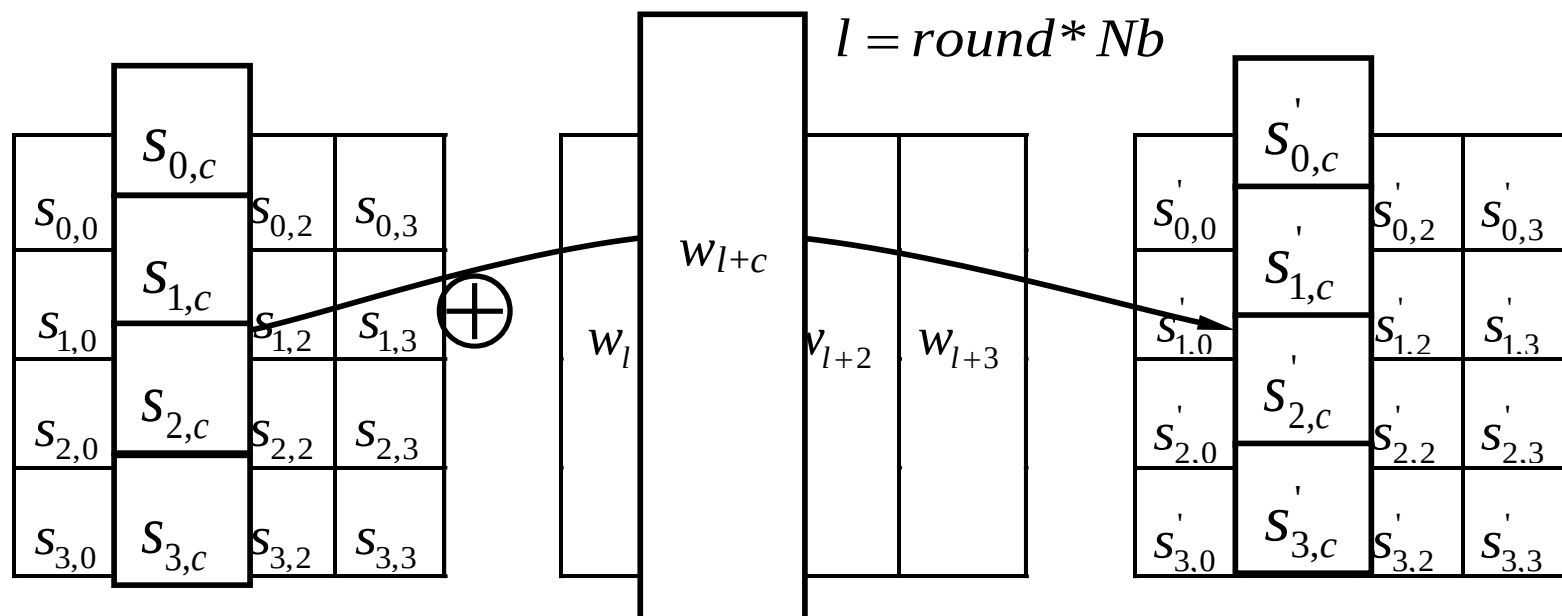
2022/3/19



## AES轮函数：密钥加

### ● 密钥加 (AddRoundKey)

- 密钥加是将轮密钥简单地与状态进行逐比特异或。
- 轮密钥由种子密钥通过密钥编排算法得到，轮密钥长度等于分组长度  $N_b$ 。



2022/3/19



## AES轮函数：密钥加

### ● 密钥加 (AddRoundKey)

- 状态State与轮密钥RoundKey的密钥加运算表示为：

AddRoundKey (State, RoundKey)

|          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|
| $a_{00}$ | $a_{01}$ | $a_{02}$ | $a_{03}$ | $a_{04}$ | $a_{05}$ |
| $a_{10}$ | $a_{11}$ | $a_{12}$ | $a_{13}$ | $a_{14}$ | $a_{15}$ |
| $a_{20}$ | $a_{21}$ | $a_{22}$ | $a_{23}$ | $a_{24}$ | $a_{25}$ |
| $a_{30}$ | $a_{31}$ | $a_{32}$ | $a_{33}$ | $a_{34}$ | $a_{35}$ |

 $\oplus$ 

|          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|
| $k_{00}$ | $k_{01}$ | $k_{02}$ | $k_{03}$ | $k_{04}$ | $k_{05}$ |
| $k_{10}$ | $k_{11}$ | $k_{12}$ | $k_{13}$ | $k_{14}$ | $k_{15}$ |
| $k_{20}$ | $k_{21}$ | $k_{22}$ | $k_{23}$ | $k_{24}$ | $k_{25}$ |
| $k_{30}$ | $k_{31}$ | $k_{32}$ | $k_{33}$ | $k_{34}$ | $k_{35}$ |

 $=$ 

|          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|
| $b_{00}$ | $b_{01}$ | $b_{02}$ | $b_{03}$ | $b_{04}$ | $b_{05}$ |
| $b_{10}$ | $b_{11}$ | $b_{12}$ | $b_{13}$ | $b_{14}$ | $b_{15}$ |
| $b_{20}$ | $b_{21}$ | $b_{22}$ | $b_{23}$ | $b_{24}$ | $b_{25}$ |
| $b_{30}$ | $b_{31}$ | $b_{32}$ | $b_{33}$ | $b_{34}$ | $b_{35}$ |

密钥加运算示意图

- 密钥加运算的逆运算是自己。

王卫华



## AES轮函数：总结

### ● AES的轮函数

- 综上所述，组成Rijndael轮函数的计算部件简洁快速，功能互补。

王华



## AES轮函数：伪码实现

### ● AES的轮函数

➤ 轮函数的伪C代码如下，前面多轮包括4个步骤：

```
Round (State, RoundKey)
{
    ByteSub (State);
    ShiftRow (State);
    MixColumn (State);
    AddRoundKey (State, RoundKey)
}
```

2022/3/19



## AES轮函数：伪码实现

### ● AES的轮函数

- 结尾轮的轮函数与前面各轮不同，将MixColumn这一步去掉：

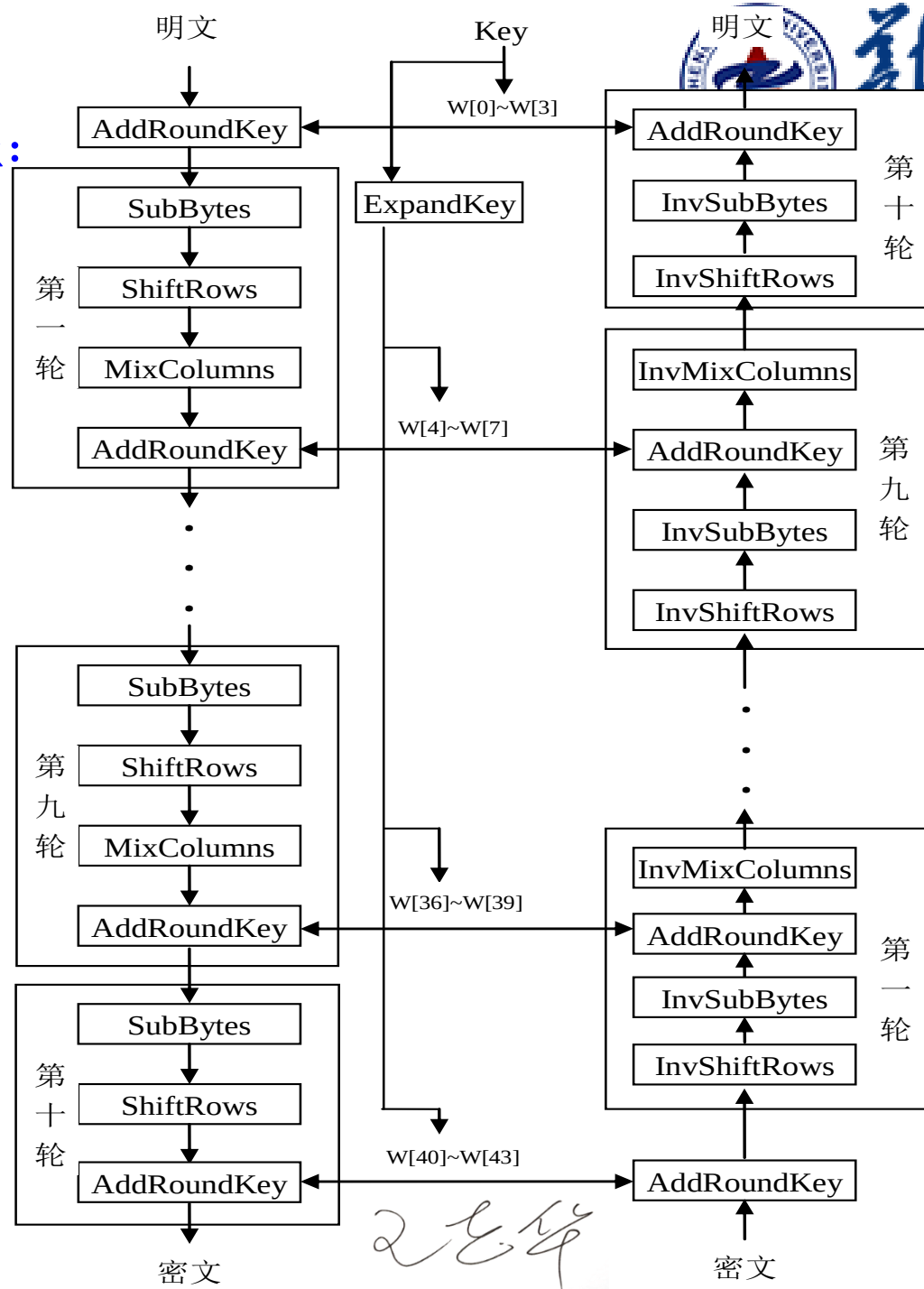
```
FinalRound (State, RoundKey)
{
    ByteSub (State);
    ShiftRow (State);
    AddRoundKey (State, RoundKey)
}
```

- 在以上伪C代码记法中，State、RoundKey 可用指针类型，“函数” Round、FinalRound、ByteSub、ShiftRow、MixColumn、AddRoundKey都在指针State、RoundKey所指向的阵列上进行运算。

王华



# AES算法过程:





## 7、AES密钥编排

### ● 密钥编排：

- 密钥编排是指从种子密钥得到**轮密钥**的过程，它由**密钥扩展**和**轮密钥选取**两部分组成。
- 密钥编排的基本原则如下：
  - ① 轮密钥的总比特数等于**分组长度**乘以**轮数加1**；
  - ② 种子密钥被扩展成为**扩展密钥**；
  - ③ **轮密钥**从**扩展密钥**中取，其中第1轮轮密钥取扩展密钥的前  $N_b$  个字，第2轮轮密钥取接下来的  $N_b$  个字，如此下去。

王卫华

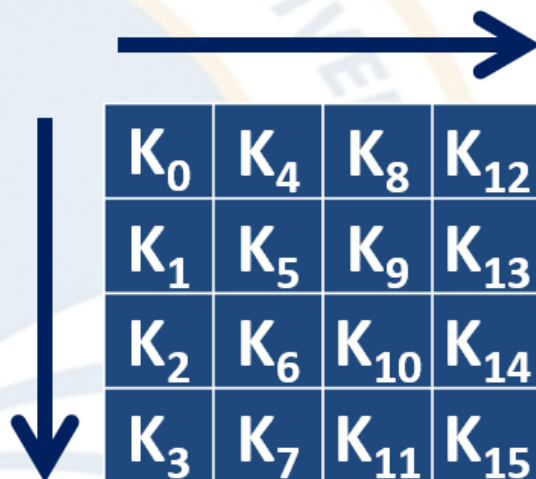


## AES密钥编排：种子密钥

### ● 密钥扩展

➤ 4个字的**种子密钥**如下：

|       |       |       |       |       |       |       |       |       |       |          |          |          |          |          |          |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|----------|----------|----------|----------|----------|----------|
| $K_0$ | $K_1$ | $K_2$ | $K_3$ | $K_4$ | $K_5$ | $K_6$ | $K_7$ | $K_8$ | $K_9$ | $K_{10}$ | $K_{11}$ | $K_{12}$ | $K_{13}$ | $K_{14}$ | $K_{15}$ |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|----------|----------|----------|----------|----------|----------|



2022/3/19



## AES密钥编排：以4个字为例

- 当分组长度和密钥长度都是**128位**时,AES的加密算法共迭代**10轮**，需要**11个**子密钥
- **AES**的密钥扩展的目的是将输入的**128位**密钥扩展成**11个128位**的子密钥
- **AES**的密钥扩展算法是以**字**为一个基本单位
- 一个字等于**4个字节**，共**32位**，刚好是密钥矩阵的一列
- 因此**4个字（128位）**密钥需要扩展成**11个**子密钥，共**44个字**

王立华



## AES密钥编排：扩展密钥

### ● 密钥扩展

- **扩展密钥**是以4个字节的**字为元素**的一维阵列，表示为 $W[N_b * (N_r + 1)]$ ，其中**前 $N_k$ 个字**取为种子密钥，以后每个字按**递归方式**定义。
- 扩展算法根据 $N_k \leq 6$ 和 $N_k > 6$ 有所不同。

王立华



## AES密钥编排：密钥扩展思路

- 若 $Nr$ 表示加密算法轮数，那么密钥扩展总共生成 $Nb(Nr+1)$ 个字
- 扩展后的密钥编排结果由一个4-byte字的线性数组组成
- 第一个子密钥的 $Nk$ 个字由密码的原始密钥直接填充
- 下面以 $Nk=4$ （即密钥为128位）为例,说明AES的密钥扩展过程

王华





## AES密钥编排：密钥扩展过程 $Nk=4$

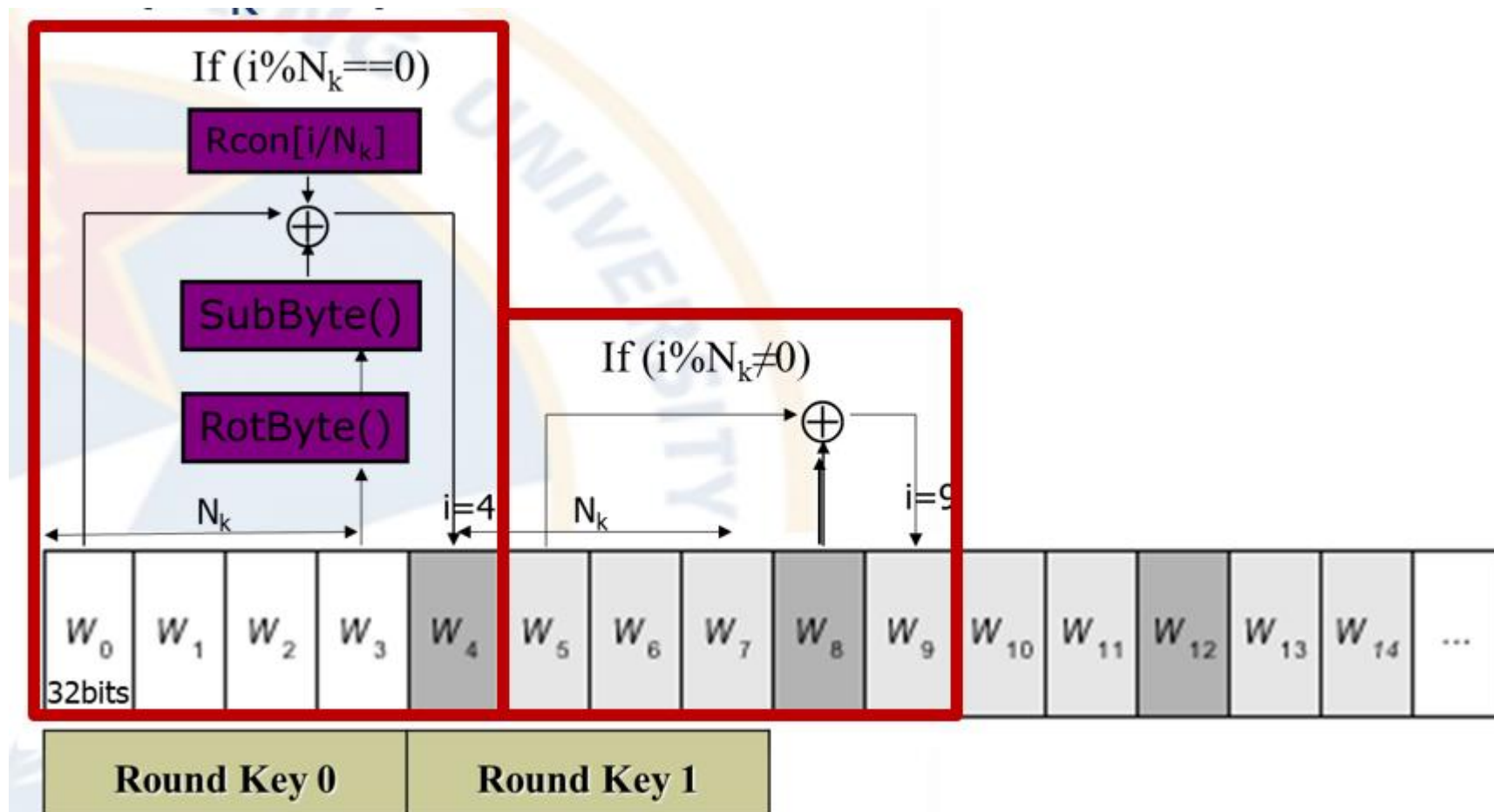
- (1) 将输入密钥直接复制到扩展密钥数组的前4个字中，得到 $w[0]$ ,  $w[1]$ ,  $w[2]$ ,  $w[3]$ 。
- (2) 然后每次用4个字填充扩展密钥数组 $w$ 的余下部分， $w[i]$ 值依赖于  $w[i-1]$ 和  $w[i-4]$ ,  $i \geq 4$ 。
- (3) 当数组 $w$ 下标不是4的倍数时， $w[i]$ 值为  $w[i-1]$ 和  $w[i-4]$  的异或。
- (4) 当数组 $w$ 下标为4的倍数时，按照下面方法计算：
  - ① 将一个字的四个字节循环左移一个字节，即将输入字 $[b_0, b_1, b_2, b_3]$ 变为 $[b_1, b_2, b_3, b_0]$ ；
  - ② 用S盒对输入字的每个字节进行字节代换；
  - ③ 将步骤①和步骤②的结果再与轮常数 $Rcon[i]$ 相异或

2022/3/19

## AES密钥编排：密钥扩展算法

### ● 密钥扩展算法

➤ 当  $N_k \leq 6$  时,







## AES密钥编排：密钥扩展算法

### ● 密钥扩展算法

➤ 当  $N_k \leq 6$  时,

若  $i \neq 0 \bmod 4$ ,  $w_i = w_{i-1} \oplus w_{i-N_k}$

若  $i \equiv 0 \bmod 4$ ,  $w_i = w_{i-N_k} \oplus \text{SubByte}(\text{RotByte}(w_{i-1})) \oplus \text{Rcon}[i/4]$

- RotByte[ ]: 循环左移一个字节
- Rcon[ ]: 轮常数

$$\text{Rcon}[j] = (\text{RC}[j], 00, 00, 00)$$

2022



## AES密钥编排：密钥扩展算法

### ● 密钥扩展算法

➤ 当  $N_k \leq 6$  时，扩展算法为：

KeyExpansion (byte Key[4\* $N_k$ ] , W[ $N_b*(N_r+1)$ ])

{

For ( $i=0; i < N_k; i++$ )

W[ $i$ ] = ( Key[4\* $i$ ], Key[4\* $i+1$ ], Key[4\* $i+2$ ], Key[4\* $i+3$ ] );

For ( $i=N_k; i < N_b*(N_r+1); i++$ )

{

temp = W[ $i-1$ ];

if ( $i \% N_k \neq 0$ )

temp = SubByte (RotByte (temp)) ^ Rcon[ $i / N_k$ ];

W[ $i$ ] = W[ $i - N_k$ ] ^ temp;

}

}

2022/3/19



## AES密钥编排：密钥扩展算法

### ● 密钥扩展算法

- 可以看出，扩展密钥的前  $N_k$  个字即为种子密钥；
- 之后扩展密钥的每个字  $W[i]$  等于前一个字  $W[i-1]$  与  $N_k$  个位置之前的字  $W[i-N_k]$  的异或；
- 不过当  $i/N_k$  为整数时，须先将前一个字  $W[i-1]$  经过以下一系列的变换：
  - 1字节的循环移位RotByte→用S盒进行变换
  - SubByte→ 异或轮常数Rcon[ $i/N_k$ ]

王华



## AES密钥编排：密钥扩展算法参数

### ● 密钥扩展算法

➤ 当  $N_k \leq 6$  时，密钥扩展算法参数为：

- ①  $\text{Key}[4 * N_k]$  为种子密钥，看作以字节为元素的一维阵列。
- ② 函数  $\text{SubByte}()$  作4字节的 S-盒变换，返回4字节字，其中每一个字节都是用 Rijndael 的 S 盒作用到输入字对应的字节得到。

2022



## AES密钥编排：密钥扩展算法参数

### ● 密钥扩展算法

➤ 当  $N_k \leq 6$  时，密钥扩展算法参数为：

- ③ 函数  $\text{RotByte}()$  也返回4字节字，该字由输入的字循环移位得到，即当输入字为  $(a, b, c, d)$  时，输出字为  $(b, c, d, a)$
- ④  $\mathbf{Rcon}[i]$  是一组4字节常向量  $[x^{i-1}, \{00\}, \{00\}, \{00\}]$ ,  $i$  从1开始。 $x^{i-1}$  是  $\text{GF}(2)[x]/m(x)$  上的多项式。

王华



## AES密钥编排：密钥扩展算法参数

### ● 密钥扩展算法

➤ 当  $N_k \leq 6$  时，密钥扩展算法参数 **Rcon[i]** 为：

④  $Rcon[i] = (rc[i], 0x00, 0x00, 0x00)$ ； $rc[i]$  对应于多项式  $x^{i-1}/m(x)$ ，可得：

$$\begin{aligned} rc[1] &= 0x01; & rc[2] &= 0x02; \\ rc[3] &= 0x04; & rc[4] &= 0x08; \\ rc[5] &= 0x10; & rc[6] &= 0x20; \\ rc[7] &= 0x40; & rc[8] &= 0x80; \\ rc[9] &= 0x1b; & rc[10] &= 0x36; \end{aligned}$$

其中：

$$m(x) = x^8 + x^4 + x^3 + x + 1$$

2024



## AES密钥编排：密钥扩展算法参数

### ● 密钥扩展算法

➤ 当  $N_k \leq 6$  时，密钥扩展算法参数  $Rcon[i]$  计算为：

### ● 密钥扩展 ( $N_k=4$ )

#### • 轮常数

| 轮数          | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 |
|-------------|----|----|----|----|----|----|----|----|----|----|
| <u>Rcon</u> | 01 | 02 | 04 | 08 | 10 | 20 | 40 | 80 | 1B | 36 |

$$RC[1] = x^0 \bmod m(x) = 01$$

⋮

$$RC[5] = x^4 \bmod m(x) = 10$$

$$RC[6] = x^5 \bmod m(x) = 20$$

⋮

$$RC[10] = x^9 \bmod m(x) = 36$$

吴志华



## AES密钥编排：密钥扩展算法

### ● 密钥扩展算法

➤ 当  $N_k > 6$  时，扩展算法为：

KeyExpansion (byte Key[4\* $N_k$ ], W[ $N_b*(N_r+1)$ ])

{

For ( $i=0$ ;  $i < N_k$ ;  $i++$ )

W[ $i$ ] = ( Key[4\* $i$ ], Key[4\* $i+1$ ], Key[4\* $i+2$ ], Key[4\* $i+3$ ] );

For ( $i=N_k$ ;  $i < N_b*(N_r+1)$ ;  $i++$ )

{

temp = W[ $i-1$ ];

if ( $i \% N_k = 0$ )

temp = SubByte (RotByte (temp)) ^ Rcon[ $i / N_k$ ];

else if ( $i \% N_k = 4$ )

temp = SubByte (temp);

W[ $i$ ] = W[ $i - N_k$ ] ^ temp;

}

2022/3/19 }





## AES密钥编排：密钥扩展算法总结

### ● 密钥扩展算法

- $N_k > 6$  与  $N_k \leq 6$  的密钥扩展算法的区别在于：
- 当  $i$  为  $N_k$  的 4 的倍数时，须先将前一个字  $W[i-1]$  经过 SubByte 变换。

2022/3/19



## AES密钥编排：轮密钥选取

### ● 轮密钥选取

➤ **轮密钥 $i$** （即第 $i$ 个轮密钥）由**轮密钥缓冲字**

$W[N_b * i]$ 到  $W[N_b * (i+1) - 1]$ 给出

|       |       |       |       |       |       |       |       |       |       |          |          |          |          |          |     |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|----------|----------|----------|----------|----------|-----|
| $W_0$ | $W_1$ | $W_2$ | $W_3$ | $W_4$ | $W_5$ | $W_6$ | $W_7$ | $W_8$ | $W_9$ | $W_{10}$ | $W_{11}$ | $W_{12}$ | $W_{13}$ | $W_{14}$ | ... |
| 轮密钥0  |       |       |       |       |       | 轮密钥1  |       |       |       |          |          | ...      |          |          |     |

$N_b=6$ 且 $N_k=4$ 时的密钥扩展与轮密钥选取

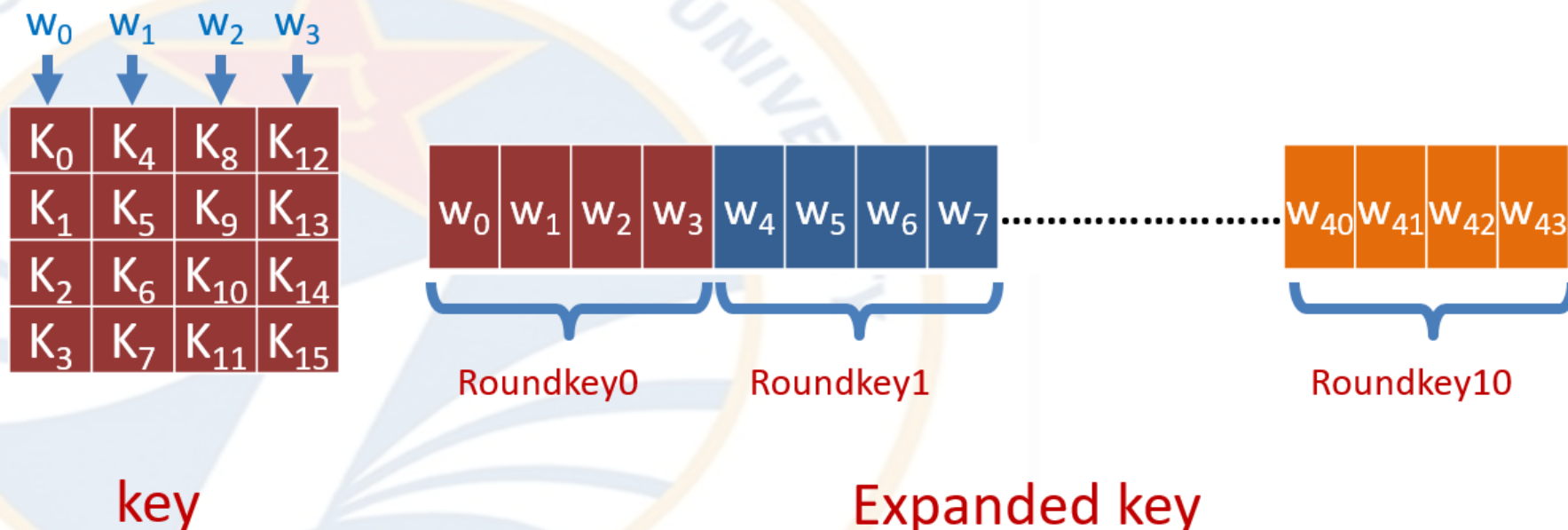
2022



## AES密钥编排：轮密钥选取

### ● 轮密钥选取

#### ● 密钥选取 ( $N_k=4$ )



2022/3/19



## 8、AES加密算法

### ● AES加密算法：

- 顺序完成以下操作： 初始的密钥加；  $(N_r-1)$ 轮迭代； 一个结尾轮。

Rijndael (State, CipherKey)

{

KeyExpansion (CipherKey, ExpandedKey);

AddRoundKey (State, ExpandedKey);

For ( $i=1; i < N_r; i++$ ) Round (State, ExpandedKey+ $N_b * i$ );

FinalRound (State, ExpandedKey+ $N_b * N_r$ )

}

- 其中， CipherKey是种子密钥， ExpandedKey是扩展密钥

2022/3/19



## AES加密算法：

### ● AES加密算法：无密钥扩展

- 密钥扩展可以事先进行（预计算），且Rijndael密码的加密算法可以用这一扩展密钥来描述：

Rijndael (State, ExpandedKey)

{

AddRoundKey (State, ExpandedKey);

For ( $i=1; i < N_r; i++$ ) Round (State, ExpandedKey+ $N_b * i$ );

FinalRound (State, ExpandedKey+ $N_b * N_r$ )

}

2022



## AES加密伪代码:

```
Cipher(byte in[4*Nb], byte out[4*Nb], word w[Nb*Nr+1])  
begin  
    byte state[4,Nb]  
    state = in  
    AddRoundKey(state, w[0, Nb-1])  
    for round = 1 step 1 to Nr-1  
        SubBytes(state)  
        ShiftRows(state)  
        MixColumns(state)  
        AddRoundKey(state, w[round*Nb, (round+1)*Nb-1])  
    end for  
    SubBytes(state)  
    ShiftRows(state)  
    AddRoundKey(state, w[Nr*Nb, (Nr+1)*Nb-1])  
    Out = state  
end
```

2024



## 9、AES解密算法

- Rijndael密码的解密算法与加密算法的计算网络都相同，只是将各计算部件换为对应的逆部件。
- 解密算法的轮函数如下：

InvRound (State, RoundKey)

{

InvByteSub (State);

InvShiftRow (State);

InvMixColumn (State);

AddRoundKey (State, RoundKey)

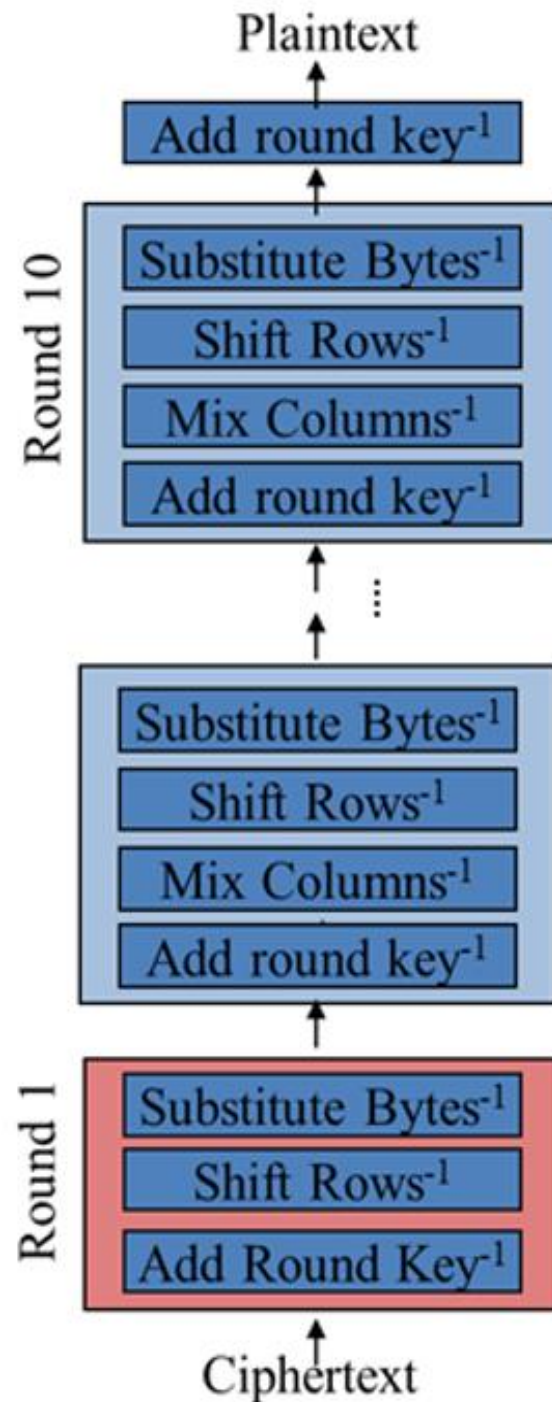
}

2022/3/19

## AES解密算法:

### ● 解密框图

- $\text{AddRoundKey}^{-1}(\text{State}, \text{Roundkey})$
- $\text{MixColumn}^{-1}(\text{State});$
- $\text{ShiftRow}^{-1}(\text{State})$
- $\text{ByteSub}^{-1}(\text{State})$



2022/3/19





## AES解密伪代码:

```
InvCipher(byte in[4*Nb], byte out[4*Nb], word w[Nb*Nr+1])  
begin  
    byte state[4,Nb]  
    state = in  
    AddRoundKey(state, w[Nr*Nb, (Nr+1)*Nb-1])  
    for round = Nr-1 step -1 downto 1  
        InvShiftRows(state)  
        InvSubBytes(state)  
        AddRoundKey(state, w[round*Nb, (round+1)*Nb-1])  
        InvMixColumns(state)  
    end for  
    SubBytes(state)  
    ShiftRows(state)  
    AddRoundKey(state, w[0, Nb-1])  
    Out = state  
end
```

~ ~ ~



## AES解密算法理论基础:

首先给出以下的结论。

**引理 3-1** 设字节代换 (ByteSub) 、行移位 (ShiftRow) 的逆变换分别为 InvByteSub、InvShiftRow。则组合部件 “ByteSub  $\rightarrow$  ShiftRow” 的逆变换为 “InvByteSub  $\rightarrow$  InvShiftRow”。

**证明** 组合部件 “ByteSub  $\rightarrow$  ShiftRow” 的逆变换原本为 “InvShiftRow  $\rightarrow$  InvByteSub”。由于字节代换 (ByteSub) 是对每个字节进行相同的变换，故 “InvShiftRow” 与 “InvByteSub” 两个计算部件可以交换顺序。

(引理3-1证毕)

王华



## AES解密算法理论基础:

**引理 3-2** 设列混合 ( MixColumn ) 的逆变换为 InvMixColumn。则列混合部件与密钥加部件 ( AddRoundKey ) 的组合部件

“MixColumn  $\rightarrow$  AddRoundKey ( $\cdot$ , Key)”

的逆变换为

“InvMixColumn  $\rightarrow$  AddRoundKey ( $\cdot$ , InvKey)”

其中密钥InvKey与Key的关系为:  $\text{InvKey} = \text{InvMixColumn}(\text{Key})$ 。

2022



## AES解密算法理论基础:

### 证明

组合部件“MixColumn→AddRoundKey (•,Key)” 的逆变换原本为

“AddRoundKey (•, Key)→InvMixColumn”

设S和K分别表示状态阵列和轮密钥阵列, 由于

$$(S \oplus K) \otimes d(x) = (S \otimes d(x)) \oplus (K \otimes d(x))$$

所以

“AddRoundKey (•, Key)→InvMixColumn”

=“InvMixColumn→AddRoundKey(•,InvMixColumn(Key))”

(引理3-2证毕)

王卫华



## AES解密算法理论基础:

**引理3-3** 将某一轮的后两个计算部件和下一轮的前两个计算部件组成组合部件, 该组合部件的程序为

```
MixColumn (State);  
AddRoundKey (State, Key(i));  
ByteSub (State);  
ShiftRow (State)
```

则该组合部件的逆变换程序为

```
InvByteSub (State);  
InvShiftRow (State);  
InvMixColumn (State);  
AddRoundKey (State, InvMixColumn (Key(i)))
```

王华



## AES解密算法理论基础:

**引理3-3证明** 这是引理3-1和引理3-2的直接推论。注意到在引理3-3所描述的逆变换中，第2步到第2步在形状上很像加密算法的轮函数，这将是解密算法的轮函数。

**定理3-2** Rijndael密码的解密算法为顺序完成以下操作：初始的密钥加； $(N_r-1)$ 轮迭代；一个结尾轮。

王华



## AES解密算法:

设加密算法的初始密钥加、第1轮、第2轮、...、第 $N_r$ 轮的子密钥依次为

$$k(0), k(1), k(2), \dots, k(N_r-1), k(N_r)$$

则解密算法的初始密钥加、第1轮、第2轮、...、第 $N_r$ 轮的子密钥依次为

$$k(N_r), \text{InvMixColumn}(k(N_r-1)), \text{InvMixColumn}(k(N_r-2)), \dots, \\ \text{InvMixColumn}(k(1)), k(0)。$$

**证明** 这是上述三个引理的直接推论。

王华



## 10、AES总结

### ● AES特点如下：

- 有可变长的加密密钥和分组长度。
- 能给出算法的最佳差分特征的概率和最佳线性逼近的偏差的界。
- 密钥安装快。
- 对RAM和ROM的要求较低。

### ● AES总结如下：

- AES没有明显的安全漏洞。
- AES在实现上有较好的灵活性和较高的效率。
- NIST宣称AES是可出口的。
- AES已经成为事实上的国际商业密码算法的标准之一。

王华





## 11、AES安全性分析

4-轮AES的最佳差分特征的概率及线性逼近的偏差分别为 $2^{-150}$ 和 $2^{-76}$ 。

8-轮AES的最佳差分特征的概率及线性逼近的偏差分别为 $2^{-300}$ 和 $2^{-151}$ 。

“Square”攻击是针对Square算法提出的一种攻击方法。AES是在Square的基础上发展起来的。但7轮以上的AES对“Square”攻击免疫的。

王华



## 11、AES安全性分析

- 对所有已知攻击而言，AES是安全的。它的设计的各个方面融合了各种特色，从而为抵抗各种攻击提供了安全性：
  - ① S盒构造中**有限域逆操作**的使用导致了线性逼近和差分分布表中的各项趋近于均匀分布。这就为抵御差分和线性攻击提供了安全性。
  - ② **线性变换 MixColumns** 使得找到包含“较少”活动S盒的差分和线性攻击成为不可能事件（设计者将这一特色称为宽轨道策略）。
  - ③ 显然，对AES现在还不存在快于**穷尽密钥搜索**的攻击。即使是对 AES 减少迭代轮数的各种变体而言“最好”的攻击，也对10轮的 AES 无效。

王卫华

# AES标准 - 思维导图

由NIST (National Institute of Standards and Technology) 征集, 共15种算法入选

共5种算法入围决赛: MARS, RC6, Rijndael, Serpent和Twofish, 经过3年的公开讨论及验证, 最终Rijndael入选

标准名称: FIPS 197  
密钥长度: 128/192/256位三种格式  
分组大小: 128位/16字节  
DES算法的替代  
全称: Advanced Encryption Standard

由荷兰学者发明: Joan Daemen和Vincent Rijmen

分组大小: 128/160/192/224/256位  
密钥长度: 128/160/192/224/256位5种格式

字节代换 (SubByte)  
行移位 (ShiftRow)  
列混合 (MixColumn)  
轮密钥加 (AddRoundKey)

迭代轮数随分组及密钥长度的不同而变化, 最低10轮, 最高14轮

根据种子密钥 (用户密钥) 生成轮密钥的过程  
对应128/192/256密钥分别生成11/13/15组128位的轮密钥

有限域GF(256)上的非线性替换  
Rijndael算法中唯一非线性部件

AES

工作模式

- ECB Electronic CodeBook mode (电子密码模式)
- CBC Cipher Block Chaining mode (密码分组链接模式)
- CFB Cipher FeedBack mode (密文反馈模式)
- OFB Output FeedBack mode (输出反馈模式)
- CTR CounTeR mode (计数器模式)
- GCM Galois Counter Mode (伽罗瓦计数器模式)

硬件加速

- Intel Advanced Encryption Standard New Instructions
  - Intel及AMD的较新的CPU均支持
- AES\_NI
  - CPU指令
    - AESENC
    - AESENCLAST
    - AESDEC
    - AESDECLAST
    - AESKEYGENASSIST
    - AESIMC
- ARM体系
- SUN SPARC T3

攻击手段

- 线性及差分分析
- 相关性分析
- 代数分析
- 立方体分析
- 电磁辐射分析
- 时序分析
- 差分故障分析
- 旁路攻击
  - 简单功耗分析 (SPA)
  - 功耗分析
    - 差分功耗攻击 (DPA)
    - 高阶DPA



## 12、DES和AES的对比

| 指标     | AES                                          | DES                  |
|--------|----------------------------------------------|----------------------|
| 明文分组长度 | 分组长度灵活，可为128bits.192bits, 256bits            | 分组长度固定为64bits,不灵活    |
| 密钥位数   | 密钥长度比DES大，并且可以在128bits.192bits和256bits之间灵活选择 | 密钥长度固定为56bits,太短且不灵活 |
| 抗攻击性   | 可对抗差分分析和线性分析                                 | 不能对抗差分和线性密码分析        |
| 效率     | 加解密速度快                                       | 加解密相对较慢              |
| 安全性    | 用穷举法是不可能破解的，穷举攻击是目前唯一                        | 已经被攻破，并且平均破解时间越来越短   |



思考:

- 编程实现GF ( $2^8$ ) 上元素的加法和乘法运算。
- 编程实现AES相关算法。

<https://blog.csdn.net/ayang1986/article/details/83104763>

<https://github.com/matt-wu/AES/>

王华